



ONLINE BANKING SYSTEM

ABOUT US GROUP A

Back-end Members:

Abanoub Abo El Saad

Donia Ahmed Nasr Ahmed

Mahmoud Sobhy Mahmoud Abdullah

Front-end Members:

Eslam Adel

Assem Sedek

Salma Ali

IOS Members:

Nancy Ayman

Ahmed Hany

The background is a gradient of blue and purple. In the center, a laptop is visible, slightly out of focus. In the top-left and bottom-right corners, there are abstract, metallic, pinkish-red shapes that look like stylized planets or moons with rings. Two small, shiny purple spheres are also present, one near the top-left shape and one near the bottom-right shape.

OUR ARCHITECTURE

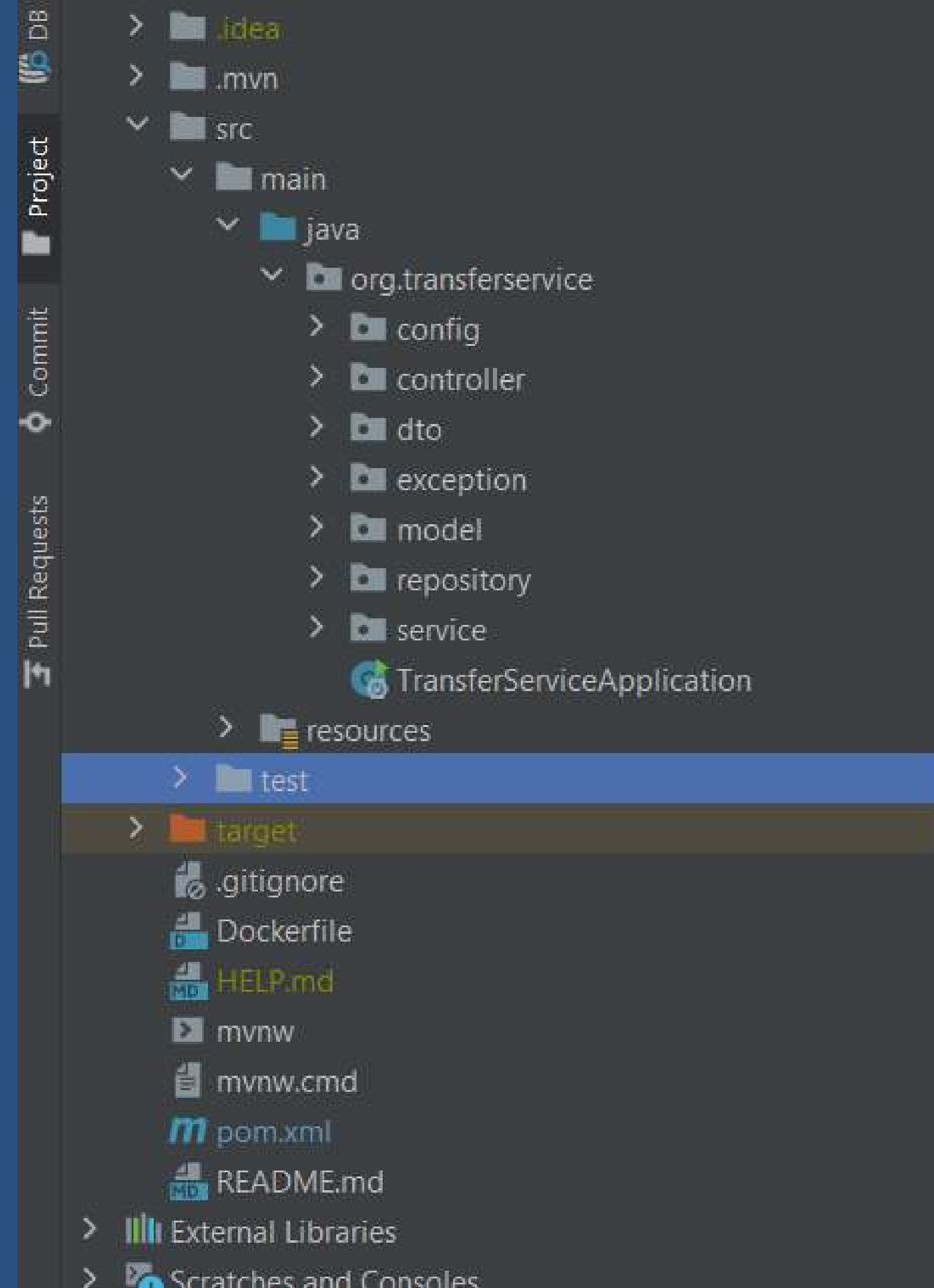
Overview of our architecture packages and what is containing:

Layered Architecture Pattern

- controller
- service
- repository

HELPER

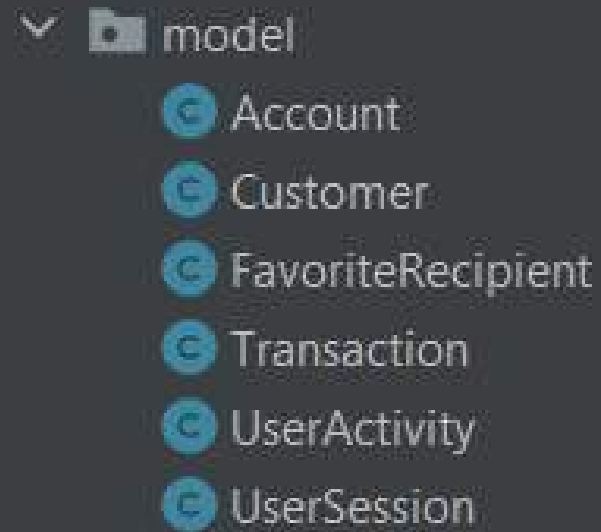
- config
- Dto
- enums
- exception
- test



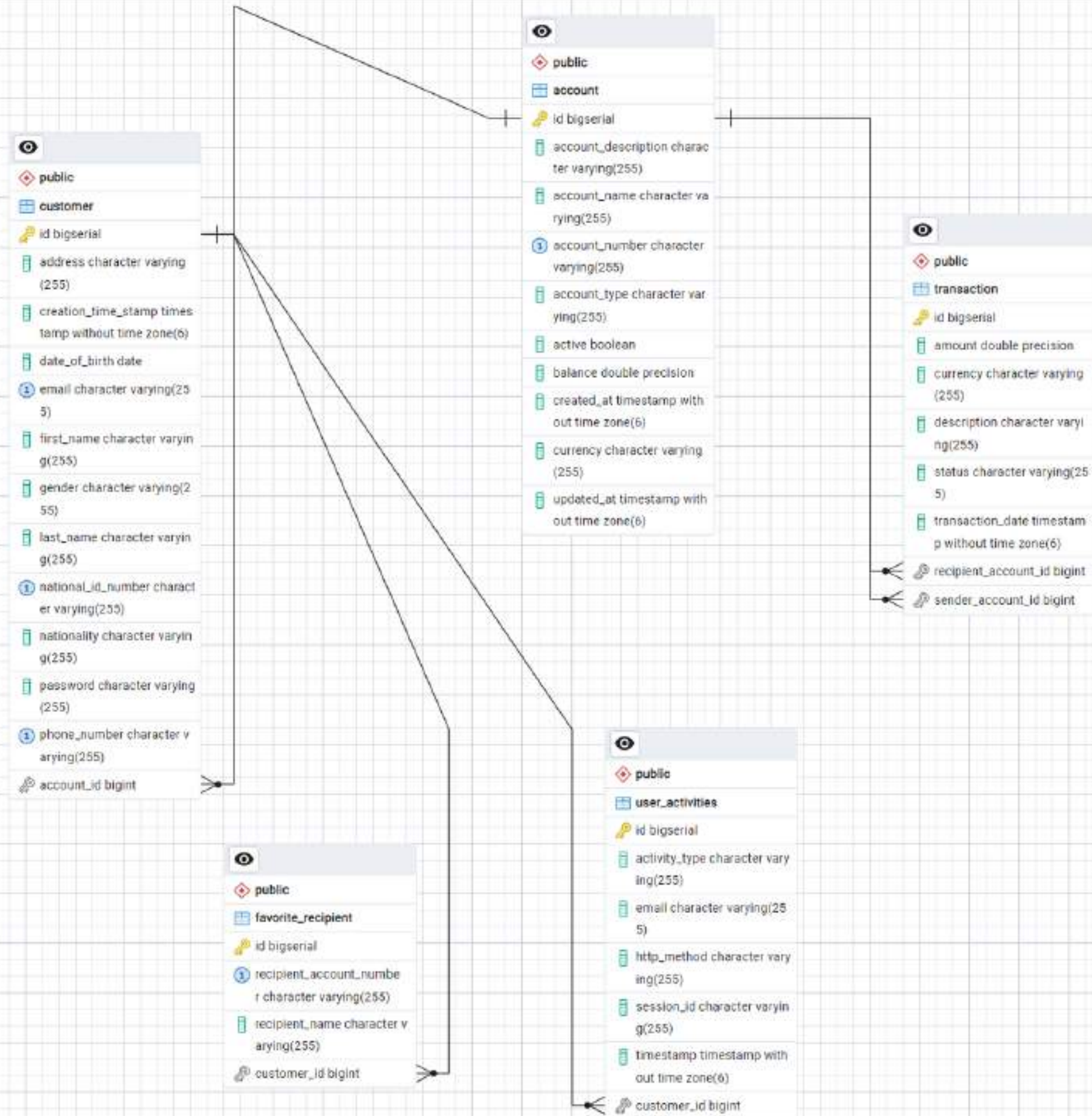
The background is a gradient of blue and purple. In the center, a laptop is visible, slightly out of focus. In the top-left and bottom-right corners, there are abstract, metallic, pinkish-red shapes that look like stylized planets or moons with rings. A small, shiny purple sphere is positioned near each of these larger shapes.

MODELS

model (use first code approach)



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.1.6.Final</version>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```



- This class carries the attributes that is related to transactions

@Entity: Marks the class as a JPA entity that will be mapped to a database table.

- @ManyToOne: This JPA annotation defines a many-to-one relationship between entities. In this case, it means that many transactions can be associated with a single account.
-
- @JoinColumn(name = "sender_account_id", nullable = false): This JPA annotation specifies the foreign key column for the senderAccount relationship. It also indicates that this column cannot be null.

```
@Data
@Entity
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class Transaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "sender_account_id", nullable = false)
    private Account senderAccount;

    @ManyToOne
    @JoinColumn(name = "recipient_account_id", nullable = false)
    private Account recipientAccount;

    @Column(nullable = false)
    private Double amount;

    @Column(nullable = false)
    private String currency;

    @CreationTimestamp
    private LocalDateTime transactionDate;

    @Column(nullable = false)
    private String status;

    private String description;

}
```

User Stories for Backend

1. *User Account Management:*

- *Account Creation:*

- Endpoint to create a new user account with name, email, and password.
- Validate and store user information securely, including password hashing.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

```
import jakarta.validation.Valid;
import org.springframework.validation.annotation.Validated;
```

```
@PostMapping("/register")
public ResponseEntity<CustomerDTO> register(@RequestBody @Valid CreateCustomerDTO createCustomerDTO) {
    CustomerDTO customerDTO = this.authenticatorService.register(createCustomerDTO);
    return new ResponseEntity<>(customerDTO, HttpStatus.CREATED);
}
```

```
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false, unique = true)
    private String phoneNumber;

    @Column(nullable = false)
    private String address;
```

New Collection / my_register(donia)

POST http://localhost:8080/api/auth/register

Params: Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Body

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "firstName": "donia",
4   "lastName": "Ahmed",
5   "email": "donia.ahmed@example.com",
6   "phoneNumber": "+9876543218",
7   "address": "789 Birch St, Oldtown, USA",
8   "nationality": "Italian",
9   "nationalIdNumber": "55443322119988",
10  "gender": "FEMALE",
11  "dateOfBirth": "1995-08-12",
12  "account": {
13    "id": 1,
14    "accountNumber": "908456662",
15    "accountType": "SAVINGS",
16    "balance": 0.0,
17    "currency": "EGP",
18    "accountName": "donia Ahmed",
19    "accountDescription": "Savings Account",
20    "active": true,
21    "createdAt": "2024-08-07T02:48:03.013052",
22    "updatedAt": "2024-08-07T02:48:03.01345"
```

Status: 200 OK Time: 1541 ms Size: 940 B

Activate Windows Go to Settings to activate Windows.

```
private final PasswordEncoder encoder;
```

```
.password(this.encoder.encode(createCustomerDTO.getPassword( )))
```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
String encode(CharSequence rawPassword);
```

Verify the encoded password obtained from storage matches the submitted raw password after it too is encoded. Returns true if the passwords match, false if they do not. The stored password itself is never decoded.

Params: `rawPassword` - the raw password to encode and match
`encodedPassword` - the encoded password from storage to compare with

password
character varying (255)

\$2a\$10\$Ry0Ckc1p7AeenSEtFD80AubyH06BlkErXN.VGYGEk906tNhE5L...

- ***Account Login:***

- Endpoint to authenticate users using email and password.
- Generate and return authentication tokens upon successful login.

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

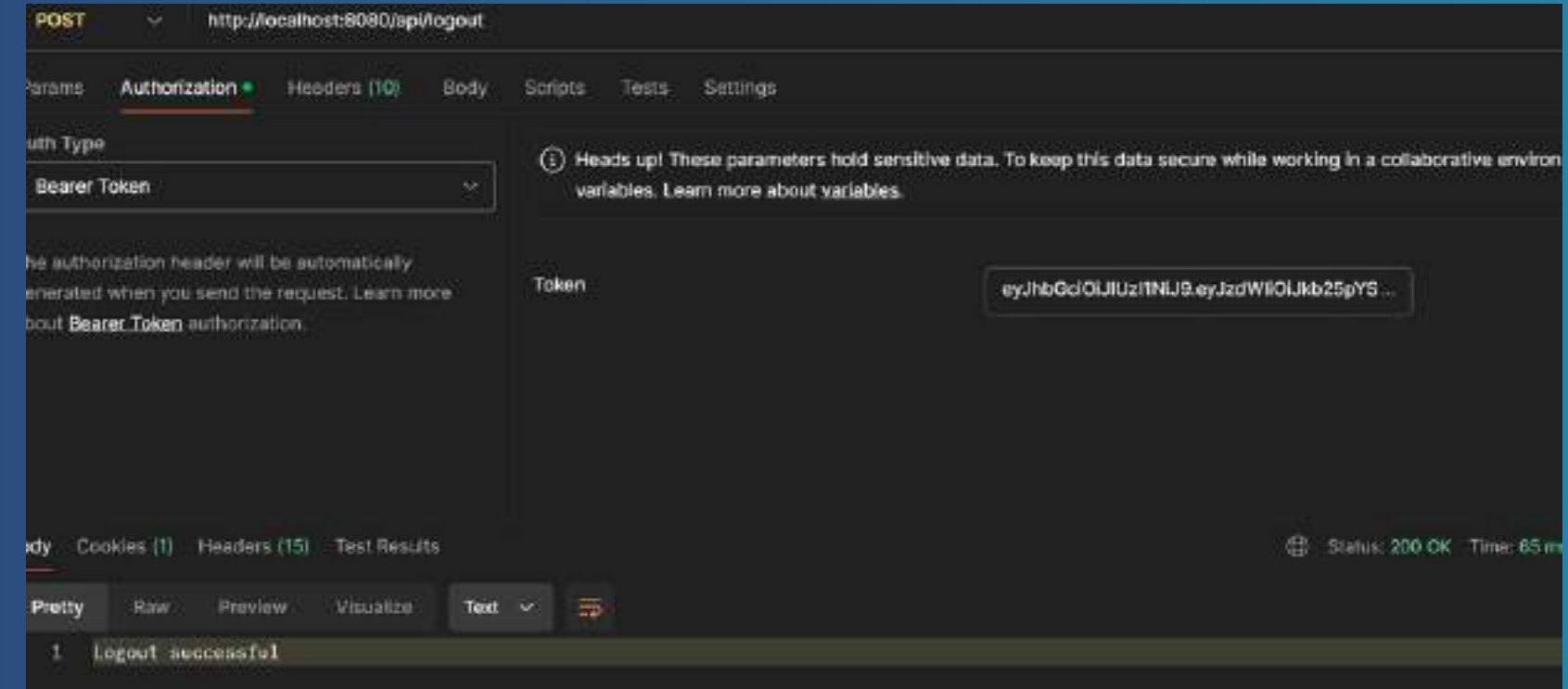
- ***Account Logout:***

- Endpoint to invalidate authentication tokens upon user logout.
- Mechanism to handle token expiration or invalidation after 30 minutes of inactivity.

```
public String generateJwtToken(Authentication authentication) {

    UserDetailsImpl userPrincipal = (UserDetailsImpl) authentication.getPrincipal();

    return Jwts.builder()
        .setSubject((userPrincipal.getUsername()))
        .setIssuedAt(new Date())
        .setExpiration(new Date((new Date()).getTime() + jwtExpirationMs))
        .signWith(key(), SignatureAlgorithm.HS256)
        .compact();
}
```



```
private Set<String> tokenBlacklist = new HashSet<>(); 1 usage
public void invalidateJwtToken(String token) { tokenBlacklist.add(token); }
```

```
@PostMapping("/logout")
public ResponseEntity<String> logout(@RequestHeader(value = "Authorization") String
    if (authorizationHeader != null && authorizationHeader.startsWith("Bearer "))
        String token = authorizationHeader.substring(beginIndex: 7);

    if (jwtUtils.validateJwtToken(token)) {
        authenticatorService.logout(token);
        return ResponseEntity.ok( body: "Logout successful");
    }
}
```

```
app.jwt.expiration.ms=1800000
```

3. *Scalability:*

- Design the backend to handle a large number of concurrent users.
- Optimize database queries and consider using caching mechanisms for frequently accessed data.

```
@Table(indexes = {
    @Index(name = "idx_customer_email", columnList = "email", unique = true),
    @Index(name = "idx_customer_phone_number", columnList = "phoneNumber", unique = true),
    @Index(name = "idx_customer_national_id_number", columnList = "nationalIdNumber", unique =
})ue)
```

Query	Query History
1	SELECT
2	indexname,
3	indexdef
4	FROM
5	pg_indexes
6	WHERE
7	tablename = 'customer';
8	

use indexing on customer Table to optimize query performance to easy retrieve the data by repository

idx_customer_email	CREATE UNIQUE INDEX idx_customer_email ON public.customer USING btree (email)
idx_customer_phone_number	CREATE UNIQUE INDEX idx_customer_phone_number ON public.customer USING btree (phone_number)
idx_customer_national_id_number	CREATE UNIQUE INDEX idx_customer_national_id_number ON public.customer USING btree (national_id_num...

SESSION MANAGMENT

2. *Session Management:*

- Implement session tracking to monitor user activity.
- Automatically invalidate sessions after 30 minutes of inactivity.

2. *Session Management:*

- Middleware to track user activity and manage session timeouts.

```
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>5.0.0</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

<dependency>
  <groupId>com.fasterxml.jackson.datatype</groupId>
  <artifactId>jackson-datatype-jsr310</artifactId>
</dependency>
```

```
@Getter
@Setter
@RedisHash("UserSession")
public class UserSession implements Serializable {

    @Id
    private String sessionId;
    private String email;
    private String activityType;
    private String httpMethod;
    private String parameters;
    private Instant timestamp;

    public UserSession(String sessionId, String email, String activityType, String httpMethod, String
parameters, Instant timestamp) {
        this.sessionId = sessionId;
        this.email = email;
        this.activityType = activityType;
        this.httpMethod = httpMethod;
        this.parameters = parameters;
        this.timestamp = timestamp;
    }
}
```

```
@Entity
@Getter
@Setter
@Table(name = "user_activities")
public class UserActivity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String sessionId;
    private String email;
    private String activityType;
    private String httpMethod;

    @Column(name = "timestamp")
    private LocalDateTime timestamp;

    public UserActivity(String sessionId, String email, String activityType, String httpMethod,
LocalDateTime timestamp) {
        this.sessionId = sessionId;
        this.email = email;
        this.activityType = activityType;
        this.httpMethod = httpMethod;
        this.timestamp = timestamp;
    }
}
```

The User Session class models It uses @Redishash for persistence in the memory as key and value, where each session being stored ain the Redis memory with the activity with TTL(Time To Live) : 30 mins

CACHING SESSION
In Memory

```
: Activity recorded in redis: Session ID: f5c50fb7-f44a-4b99-b4f2-6b5258f6e5f7, Email: donia.doe@example.com, Activity: LOGIN
: Session TTL checked: Session ID: f5c50fb7-f44a-4b99-b4f2-6b5258f6e5f7, TTL (minutes): 29
```

The UserActivity class is a JPA entity that maps to the "user_activities" table in a relational database. It stores information about user activities, This entity can be used to log and audit user actions within an application, providing valuable data for monitoring and analysis

SAVE USER ACTIVITY IN NORMAL DATABASE

	id [PK] bigint	activity_type character varying (255)	email character varying (255)	http_method character varying (255)	session_id character varying (255)	timestamp timestamp without time zone
1	1	LOGIN	john.doe@example.com	POST	9952A6F53B9C0CFC97FD656D3A80D886	2024-08-05 04:47:51.618

```

public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf(AbstractHttpConfigurer::disable)
        .exceptionHandling(e -> e.authenticationEntryPoint(unauthorizedHandler))
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
            .sessionFixation().newSession()
        )
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/register", "/api/login", "/", "/h2-console/**",
                "/actuator/**", "/swagger-ui/**", "/v3/api-docs/**", "/swagger-resources/**", "/webjars/**").permitAll()
            .anyRequest().authenticated());

    http.authenticationProvider(authenticationProvider());

    http.addFilterBefore(authenticationJwtTokenFilter(), UsernamePasswordAuthenticationFilter.class);
    http.addFilterBefore(userActivityFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

```

```

protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain fi
    throws ServletException, IOException {
    String requestURI = request.getRequestURI();
    boolean isLoginRequest = requestURI.contains("/api/login");
    HttpSession session = null;
    if (isLoginRequest) {
        session = request.getSession( create: true);
        logger.info("Creating a new session with ID: {}", session.getId());
    } else {
        session = request.getSession( create: false);
        if (session != null) {
            logger.info("Using existing session with ID: {}", session.getId());
        }
    }
}

```

```

try {
    userActivityService.checkSessionExpiration(sessionId);
    if (session != null) {
        userActivityService.recordActivity(sessionId, email != null ? email : "register still no se
    }
} catch (Exception e) {
    logger.error("Error recording user activity", e);
}

```

```

public void checkSessionExpiration(String sessionId) { 1 usage  👤 abanoub *
    // Check TTL (Time To Live)
    Long ttl = redisTemplate.getExpire(sessionId, TimeUnit.MINUTES);
    if (ttl != null) {
        if (ttl <= 0) {
            // Session has expired
            userSessionRepository.deleteById(sessionId);
            redisTemplate.delete(sessionId); // Remove from Redis as well
            logger.info("Session expired and removed: Session ID: {}", sessionId);
        } else {
            logger.info("Session TTL checked: Session ID: {}, TTL (minutes): {}", sessionId, ttl);
        }
    } else {
        logger.warn("Unable to check TTL for Session ID: {}", sessionId);
    }
}

```

```

public void recordActivity(String sessionId, String email, String activityType, String httpMethod, String parameters) { 1 usage
    Instant now = Instant.now();
    UserSession session = new UserSession(sessionId, email, activityType, httpMethod, parameters, now);

    UserActivity activity = new UserActivity(
        sessionId,
        email,
        activityType,
        httpMethod,
        LocalDateTime.now()
    );

    userActivityRepository.save(activity);
    logger.info("Activity recorded in database : Session ID: {}, Email: {}, Activity: {}", sessionId, email, activityType);
    redisTemplate.opsForValue().set(sessionId, activityType, REDIS_EXPIRY_DURATION, TimeUnit.MINUTES);
}

```

```
@Configuration
public class RedisConfig {

    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory connectionFactory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(connectionFactory);

        ObjectMapper objectMapper = new ObjectMapper();
        objectMapper.registerModule(new JavaTimeModule());

        GenericJackson2JsonRedisSerializer jackson2JsonRedisSerializer = new
        GenericJackson2JsonRedisSerializer(objectMapper);

        template.setKeySerializer(new StringRedisSerializer());
        template.setValueSerializer(jackson2JsonRedisSerializer);
        template.setHashKeySerializer(new StringRedisSerializer());
        template.setHashValueSerializer(jackson2JsonRedisSerializer);

        return template;
    }
}
```

this class configures a RedisTemplate to connect to a Redis memory and handle JSON serialization of objects to save in it as it send as bytes

THIS AT THE BEGINNING OF THE SESSION CREATION SAVED IN REDIS AND IT STORED IN THE CACHE MEMORY

```
C:\Program Files\Redis\redis-  ×  +  v
127.0.0.1:6379> keys "transferServiceSession:*"
1) "transferServiceSession:sessions:f5c50fb7-f44a-4b99-b4f2-6b5258f6e5f7"
2) "transferServiceSession:default"
127.0.0.1:6379> |
```

```
private static final long REDIS_EXPIRY_DURATION = 30;

redisTemplate.opsForValue().set(sessionId, session, REDIS_EXPIRY_DURATION, TimeUnit.MINUTES);
```

THIS AFTER 30 MINS THE SESSION TERMINATED FROM CACHE MEMORY DUE TO INACTIVITY

```
127.0.0.1:6379> keys "transferServiceSession:*"
1) "transferServiceSession:default"
127.0.0.1:6379> |
```

DEPLOYMENT

1. ***Security:***

- Implement secure authentication and authorization mechanisms.
- Use HTTPS to encrypt data in transit.
- Ensure proper validation and sanitization of all input data to prevent SQL injection and other security threats.

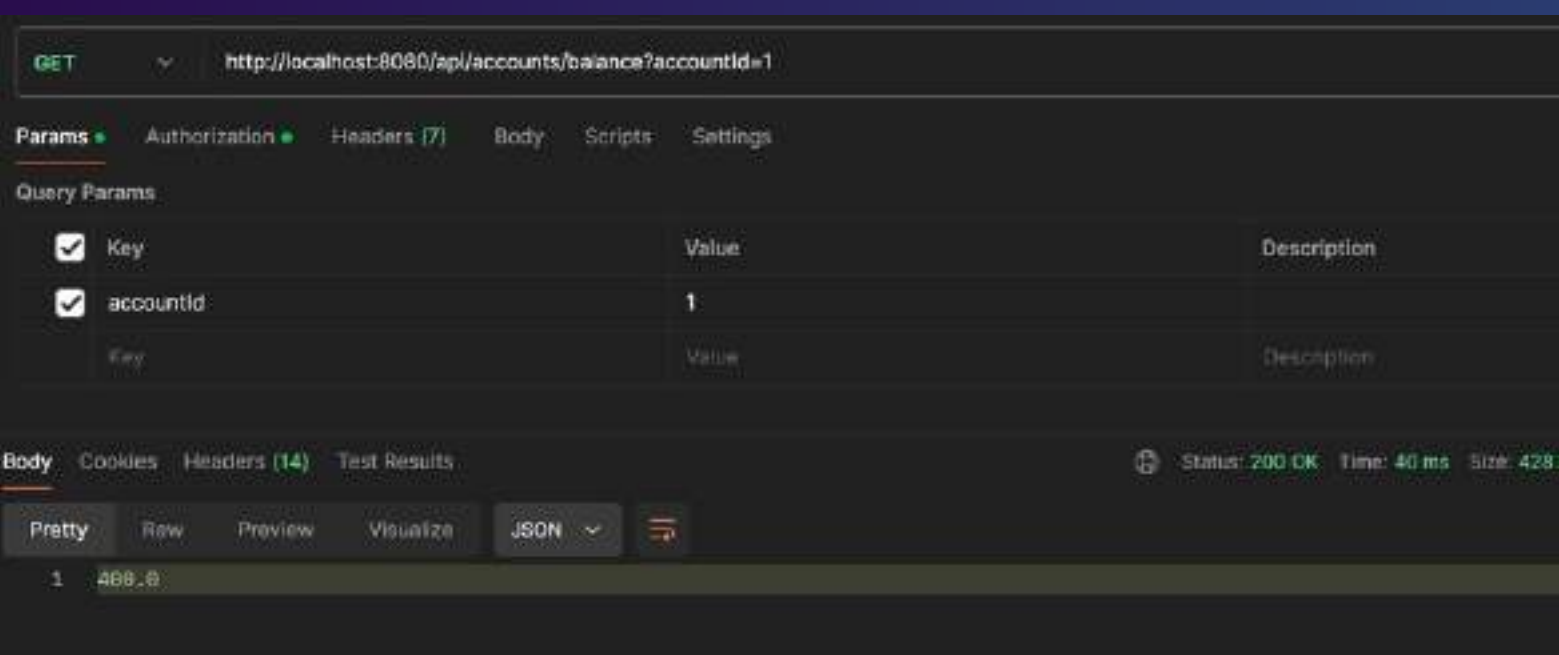
WE USED NAGROK FOR DEPLOYMENT

which enable us to use https by using nagrok ngrok handles the SSL/TLS handshake needed to manage trusted certificate and by that we do the security encryption between the network transfer

```
Forwarding      https://62b9-156-223-19-253.ngrok-free.app -> http://localhost:8080
Connections    ttl      opn      rt1      rt5      p50      p90
                0        0        0.00     0.00     0.00     0.00
```

account balance

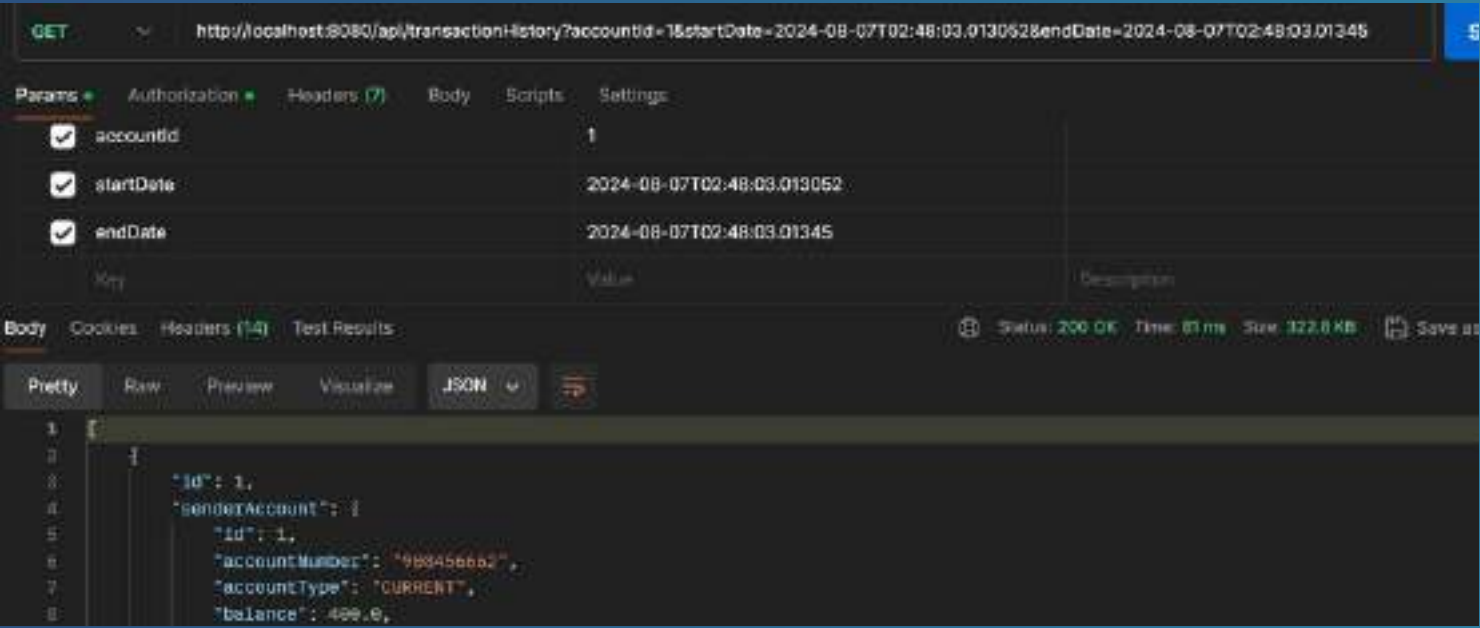
- Endpoint to fetch the current balance of the user's account.
- Secure access control to ensure only authenticated users can access their balance.



```
public Double getAccountBalance(Long id){  
    Optional<Account> accountopt = accountRepository.findById(id);  
    if(accountopt.isPresent()){  
        return accountopt.get().getBalance();  
    }  
    else {  
        throw new IllegalArgumentException("Account not found");  
    }  
}
```

4. *Transaction History:*

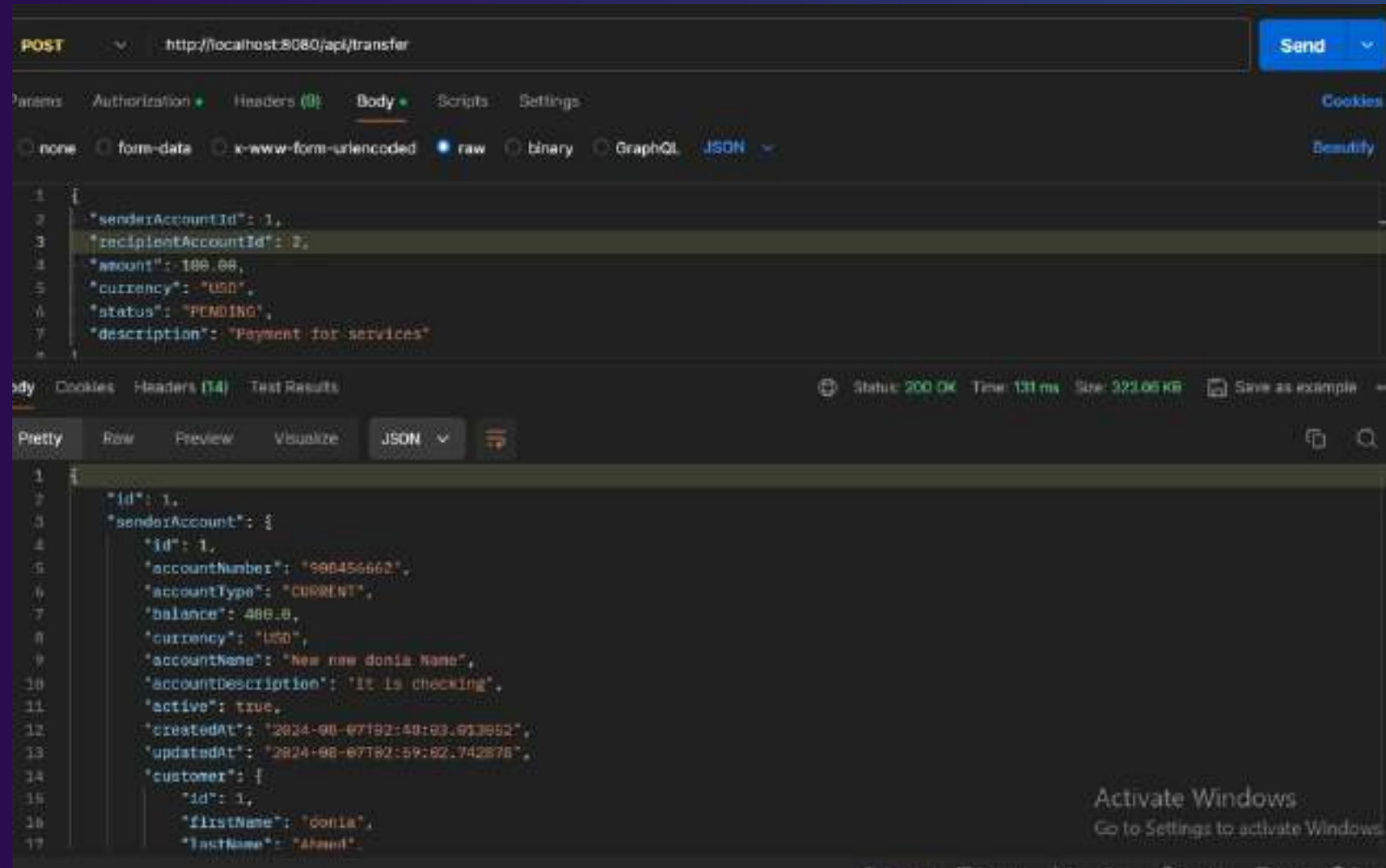
- Endpoint to retrieve the transaction history for a user.
- Support pagination or date-based filtering for efficient data retrieval.



```
public List<Transaction> getTransactionHistory(Long accountId, LocalDateTime startDate, LocalDateTime endDate)  
{  
    if (startDate == null && endDate == null) {  
        return transactionRepository.findBySenderIdOrRecipientAccountId( accountId, accountId);  
    } else {  
        return transactionRepository.findBySenderIdOrRecipientAccountIdAndTransactionDateBetween(  
            accountId, accountId, startDate, endDate);  
    }  
}
```

5. *Money Transfer:*

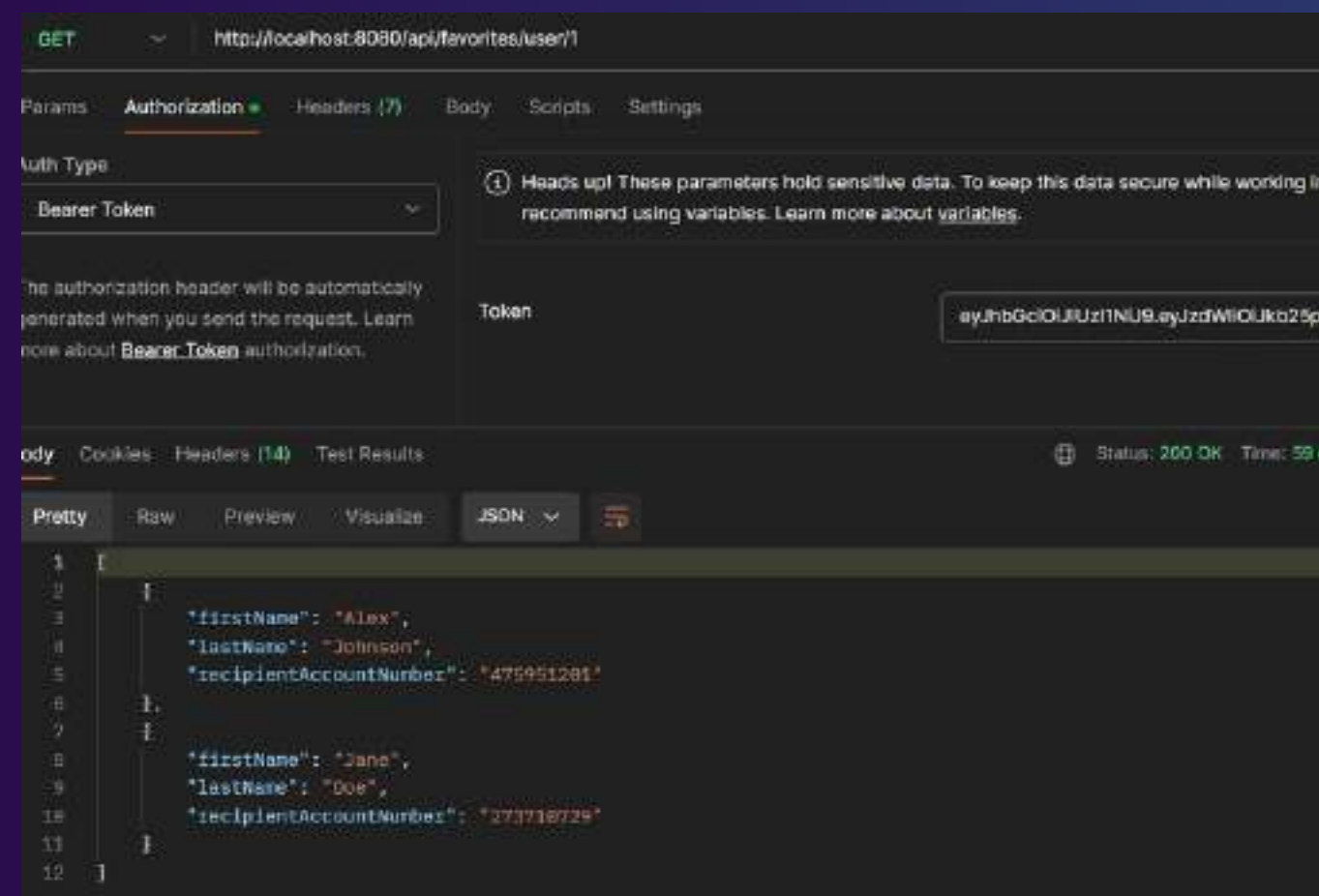
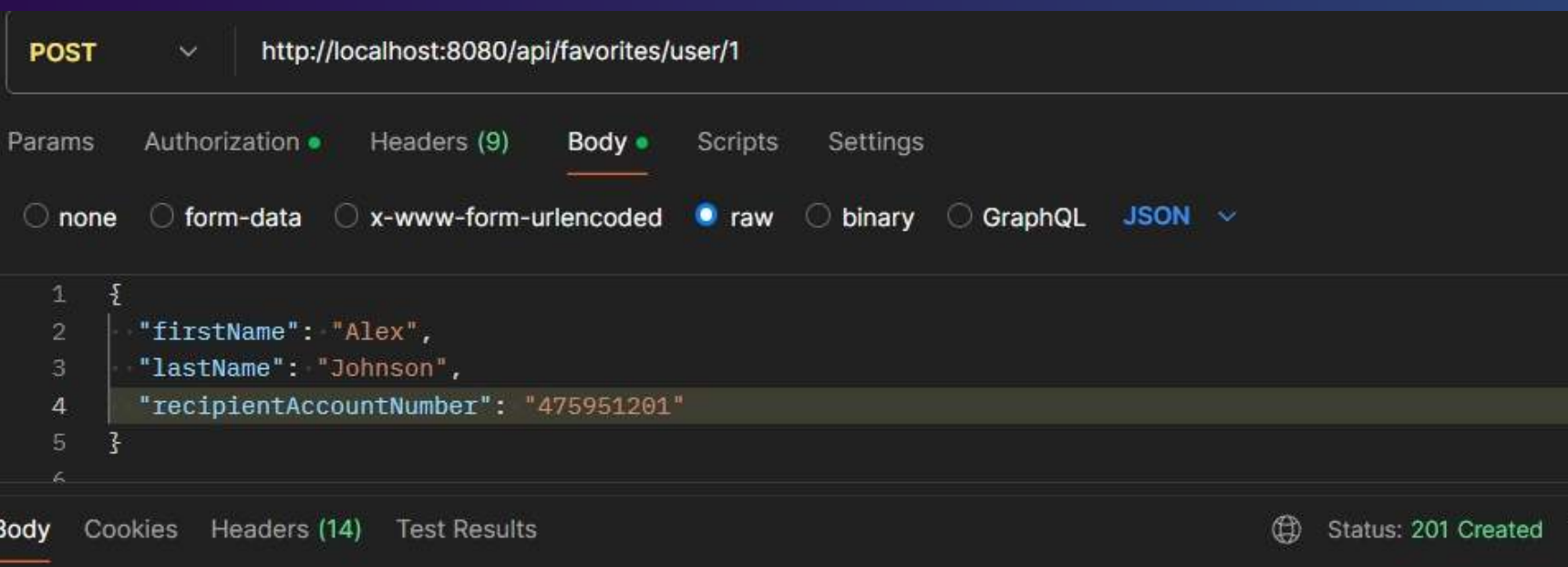
- Endpoint to initiate a money transfer to another user.
- Validate recipient details and ensure sufficient funds before processing.
- Record the transaction details and update account balances accordingly.



```
@Transactional
public Transaction createTransaction(TransactionDTO transactionDTO) {
    Optional<Account> senderAccountOpt = accountService.getAccountById(transactionDTO.getSenderAccountId());
    Optional<Account> recipientAccountOpt = accountService.getAccountById(transactionDTO.getRecipientAccountId());
    if (senderAccountOpt.isPresent() && recipientAccountOpt.isPresent()) {
        Account senderAccount = senderAccountOpt.get();
        Account recipientAccount = recipientAccountOpt.get();
        if (senderAccount.getBalance() < transactionDTO.getAmount()) {
            throw new IllegalArgumentException("Insufficient balance in sender's account");
        }
        senderAccount.setBalance(senderAccount.getBalance() - transactionDTO.getAmount());
        recipientAccount.setBalance(recipientAccount.getBalance() + transactionDTO.getAmount());
        accountRepository.save(senderAccount);
        accountRepository.save(recipientAccount);
        Transaction transaction = Transaction.builder()
            .senderAccount(senderAccountOpt.get())
            .recipientAccount(recipientAccountOpt.get())
            .amount(transactionDTO.getAmount())
            .currency(transactionDTO.getCurrency())
            .status(transactionDTO.getStatus())
            .description(transactionDTO.getDescription())
            .build();
        return transactionRepository.save(transaction);
    } else {
        throw new IllegalArgumentException("Invalid account IDs");
    }
}
```

6. *Favorite Recipients:*

- (Optional) Endpoint to save and manage favorite recipients for a user.
- Allow retrieval of favorite recipients for quick access during money transfers.

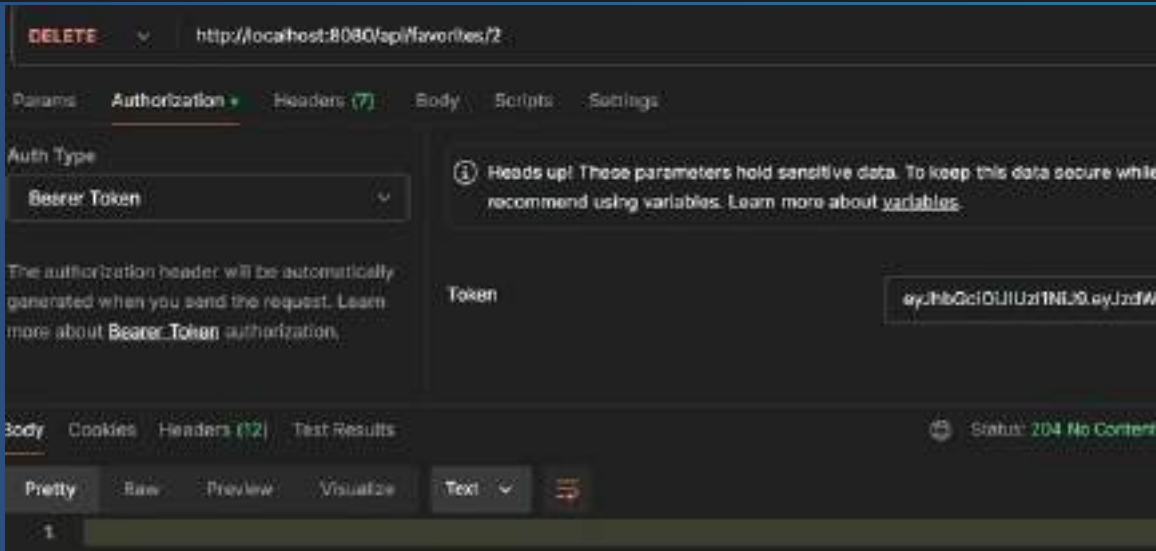


```
@Transactional (usage: 1 abanoub)
public FavoriteRecipient addFavoriteRecipient(FavoriteRecipientDTO favoriteRecipientDTO , Long customerId) {
    String recipientName = favoriteRecipientDTO.getFirstName() + " " + favoriteRecipientDTO.getLastName()
    Customer customer = customerRepository.findById(customerId)
        .orElseThrow(() -> new RuntimeException("Customer not found"));
    FavoriteRecipient favoriteRecipient = FavoriteRecipient.builder()
        .recipientName(recipientName)
        .recipientAccountNumber(favoriteRecipientDTO.getRecipientAccountNumber())
        .customer(customer)
        .build();

    return favoriteRecipientRepository.save(favoriteRecipient);
}

Explain | Test | Document | Fix | Ask
public List<FavoriteRecipientDTO> getFavoriteRecipientsByCustomerId(Long customerId) { (usage: 1 abanoub)
    List<FavoriteRecipient> favoriteRecipients = favoriteRecipientRepository.findByCustomerId(customerId)

    return favoriteRecipients.stream() Stream<FavoriteRecipient>
        .map(recipient -> {
            String[] nameParts = recipient.getRecipientName().split( regex: " ", limit: 2);
            String firstName = nameParts[0];
            String lastName = nameParts.length > 1 ? nameParts[1] : "";
            return FavoriteRecipientDTO.builder()
                .firstName(firstName)
                .lastName(lastName)
                .recipientAccountNumber(recipient.getRecipientAccountNumber())
                .build();
        }) Stream<FavoriteRecipientDTO>
        .collect(Collectors.toList());
}
```



```
@Transactional (usage: 1 abanoub)
public void removeFavoriteRecipient(Long id) { favoriteRecipientRepository.deleteById(id); }
```

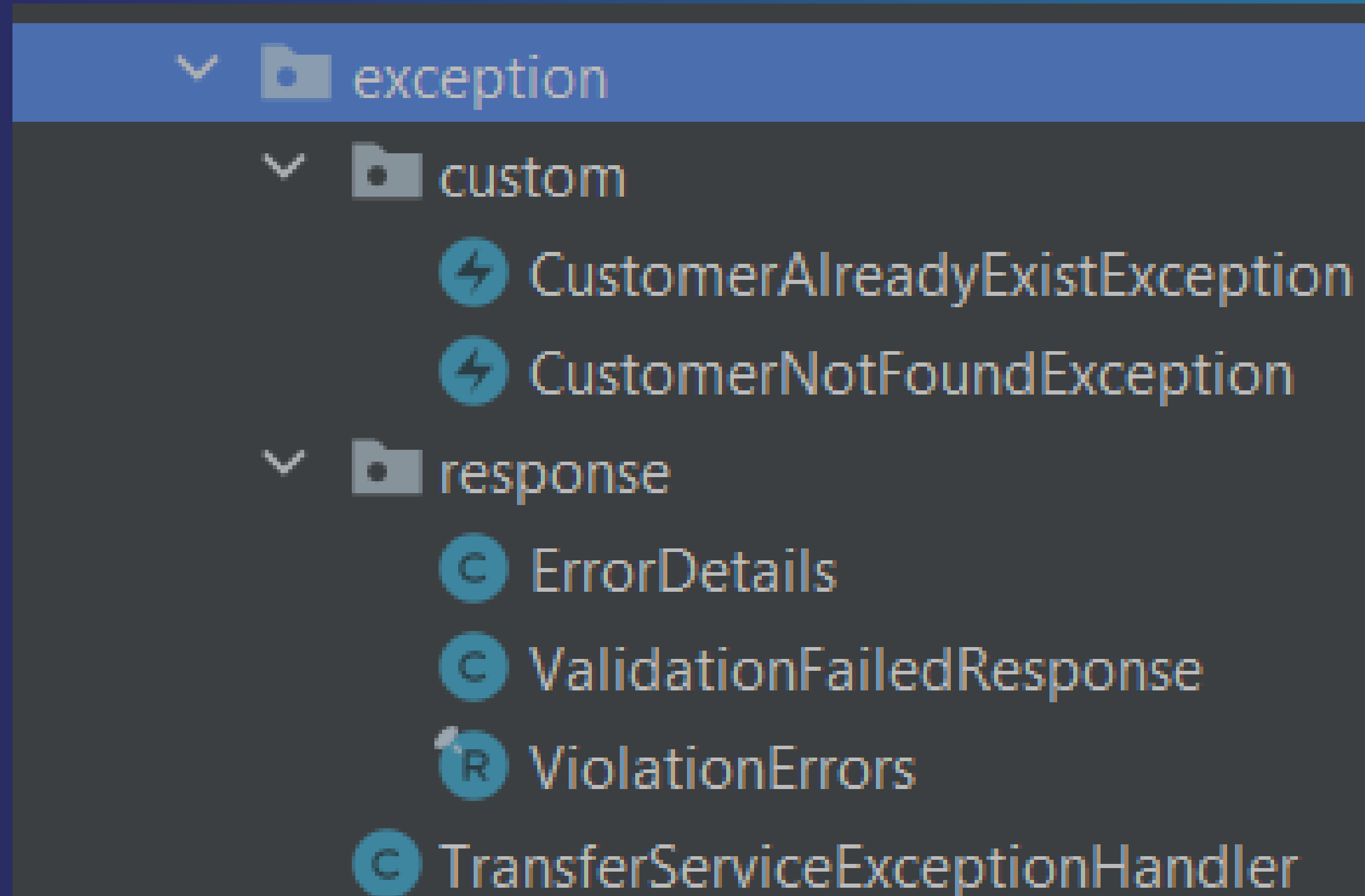
2. ***Data Integrity:***

- Ensure atomicity of money transfer operations to
- Implement proper error handling and logging.

EXCEPTIONS

The background of the slide is a gradient of blue and purple. In the lower right corner, there is a 3D rendered splash of red liquid with a small purple sphere above it. A laptop is faintly visible in the background, its screen and keyboard area appearing as a darker blue shape.

- We make a base exception handler that contains the common exception handling in the project
- Separating the others into two packages in custom and response



@ControllerAdvice: This annotation allows the class to handle exceptions across the whole application in a centralized manner and It ensures that exception handling logic is separated from the controllers.

```
@ControllerAdvice
public class TransferServiceExceptionHandler {

    @ExceptionHandler(CustomerAlreadyExistException.class)
    public ResponseEntity<Object> customerAlreadyExistExceptionHandler(CustomerAlreadyExistException
exception, WebRequest request) {
        return new ResponseEntity<>(new ErrorDetails(LocalDate.now(), exception.getMessage(),
request.getDescription(false), HttpStatus.BAD_REQUEST), HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(CustomerNotFoundException.class)
    public ResponseEntity<Object> customerNotFoundExceptionHandler(CustomerNotFoundException
exception, WebRequest request) {
        return new ResponseEntity<>(new ErrorDetails(LocalDate.now(), exception.getMessage(),
request.getDescription(false), HttpStatus.NOT_FOUND), HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Object>
methodArgumentNotValidExceptionHandler(MethodArgumentNotValidException exception) {

        ValidationFailedResponse error = ValidationFailedResponse
            .builder()
            .httpStatus(HttpStatus.BAD_REQUEST)
            .timestamp(LocalDate.now())
            .build();

        for (FieldError fieldError : exception.getBindingResult().getFieldErrors()) {

            error.getViolations().add(ViolationErrors
                .builder()
                .fieldName(fieldError.getField())
                .message(fieldError.getDefaultMessage())
                .build());
        }

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}
```

- Single Responsibility Principle :
 - Adherence: The TransferServiceExceptionHandler adheres to SRP by focusing solely on handling exceptions and generating appropriate error responses. Each method has a single responsibility related to handling a specific type of exception.
- Open/Closed Principle :
 - The class is open for extension (e.g., adding new exception handlers) but closed for modification. You can extend its functionality by adding new methods without changing existing ones.
- Exception Handling Pattern:
 - The class implements the Exception Handling pattern by providing a centralized way to handle different types of exceptions and generate appropriate HTTP responses
- Builder Pattern:
 - The ValidationFailedResponse and ViolationErrors objects use the Builder pattern (via Lombok's @Builder annotation) to create complex objects in a readable and maintainable way.

4. ***Testing:***

- Write unit tests and integration tests for all endpoints and functionalities.

UNIT TESTING

The background of the slide is a gradient of dark blue and purple. In the center, there is a faint, semi-transparent image of a laptop. In the bottom right corner, there is a 3D-rendered splash of red liquid with a small red sphere above it, creating a dynamic and modern aesthetic.

Testing for all the services and the endpoints apis

- starting with AccountServiceTest

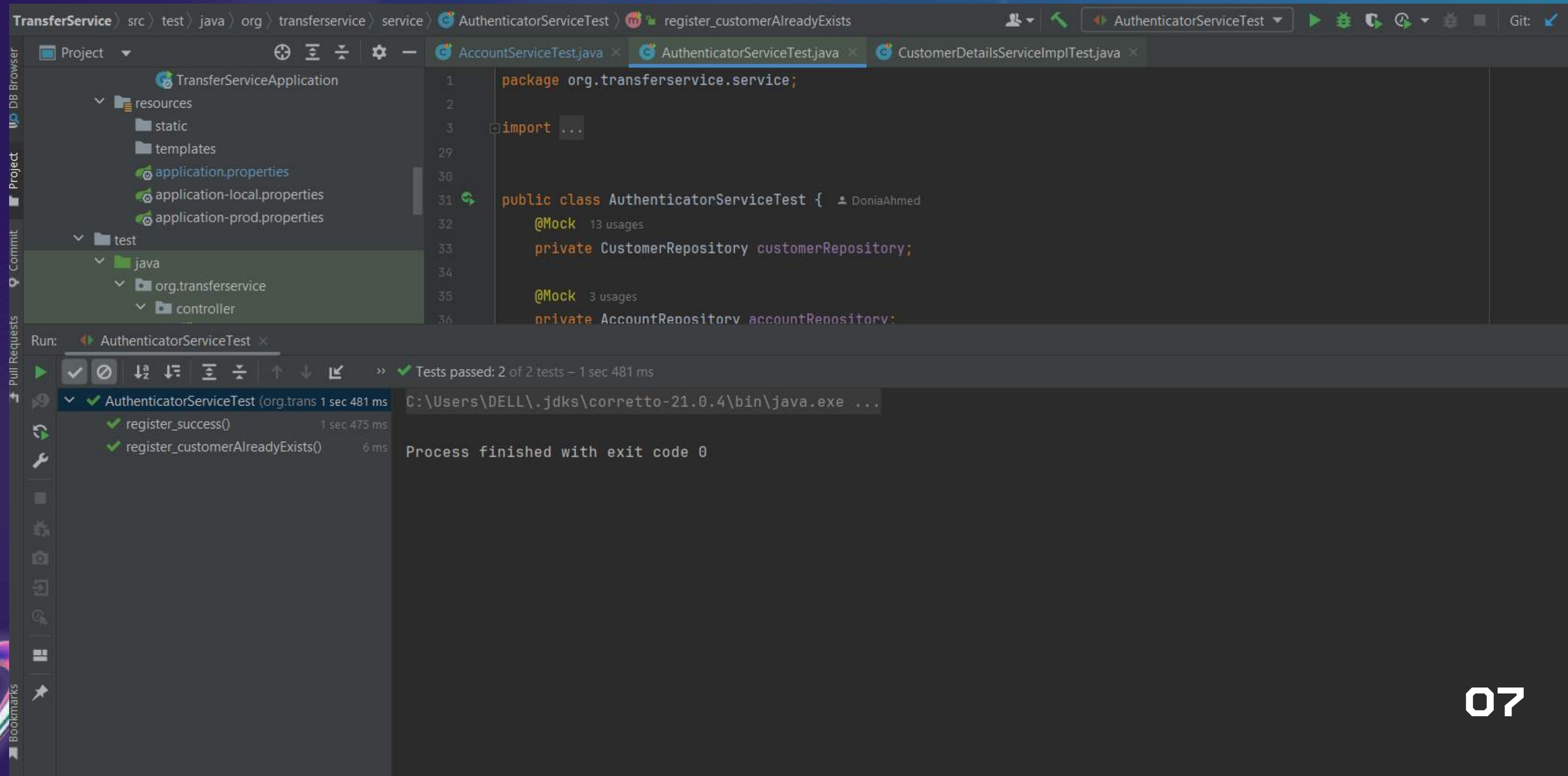
```
src > test > java > org > transferservice > service > AccountServiceTest
AccountServiceTest.java
1 package org.transferservice.service;
2
3 import ...
17
18 @ExtendWith(MockitoExtension.class) DoniaAhmed
19 public class AccountServiceTest {
20
21     @Mock 14 usages
22     private AccountRepository accountRepository;
23
24     @InjectMocks 7 usages
25     private AccountService accountService;
26
27     private Account account; 7 usages
28
29     @BeforeEach DoniaAhmed
30     void setUp() {
31         account = Account.builder()
32             .id(1L)
33             .accountNumber("123456789")
34             .balance(1000.0)
35             .accountName("Alex's Account")
36             .accountDescription("Personal Savings Account")
37             .build();
38     }
39
40     @Test DoniaAhmed
41     void createAccount_success() {
42         when(accountRepository.save(any(Account.class))).thenReturn(account);
43     }
44 }
```

- As we see all the test cases has passed successfully

The screenshot displays an IDE interface with the following components:

- Project Explorer:** Shows a project structure with folders like `security`, `resources`, and `static`. The `AccountServiceTest` class is highlighted under the `service` package.
- Code Editor:** Displays the source code of `AccountServiceTest.java`. The code includes package declarations, imports, and annotations like `@ExtendWith(MockitoExtension.class)`, `@Mock`, and `@InjectMocks`.
- Run Console:** Shows the execution results of the test class. It indicates that all 7 tests passed successfully within a total time of 1 second and 190 milliseconds. The tests listed are:
 - `updateAccount_success()` (1 sec 148 ms)
 - `getAccountById_success()` (11 ms)
 - `getAccountBalance_success()` (5 ms)
 - `deleteAccount_success()` (7 ms)
 - `createAccount_success()` (4 ms)
 - `getAccountById_notFound()` (8 ms)
 - `getAccountBalance_accountNotFound()` (7 ms)
- Output Console:** Shows the command used to run the tests: `C:\Users\DELL\.jdk\corretto-21.0.4\bin\java.exe ...` and the message: `Process finished with exit code 0`.

- The test cases of the AuthenticatorService has passed successfully as we see



- The test cases of the CustomerDetailsServiceImpl has passed successfully as we see

The screenshot shows an IDE with the following components:

- Project Explorer:** Displays the project structure. The 'test' folder is expanded, showing the 'java' package, 'org.transferservice' package, and 'controller' package. The 'service' package is also visible.
- Editor:** Shows the code for `CustomerDetailsServiceImplTest.java`. The code includes package declarations, imports, and test methods. The `setUp()` method uses `MockitoAnnotations.openMocks(this)` to initialize mocks.
- Run Console:** Shows the execution results of the tests. The output indicates that 2 out of 2 tests passed, with a total execution time of 1 second and 122 milliseconds. The tests are `loadUserByUsername_existingU` and `loadUserByUsername_userNotFound()`.

```
package org.transferservice.service;

import ...

public class CustomerDetailsServiceImplTest {
    @Mock
    private CustomerRepository customerRepository;

    @InjectMocks
    private CustomerDetailsServiceImpl customerDetailsService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
    }
}
```

Run: CustomerDetailsServiceImplTest

Tests passed: 2 of 2 tests – 1 sec 122 ms

CustomerDetailsServiceImplTest (0 1 sec 122 ms)

- loadUserByUsername_existingU 1 sec 115 ms
- loadUserByUsername_userNotFound() 7 ms

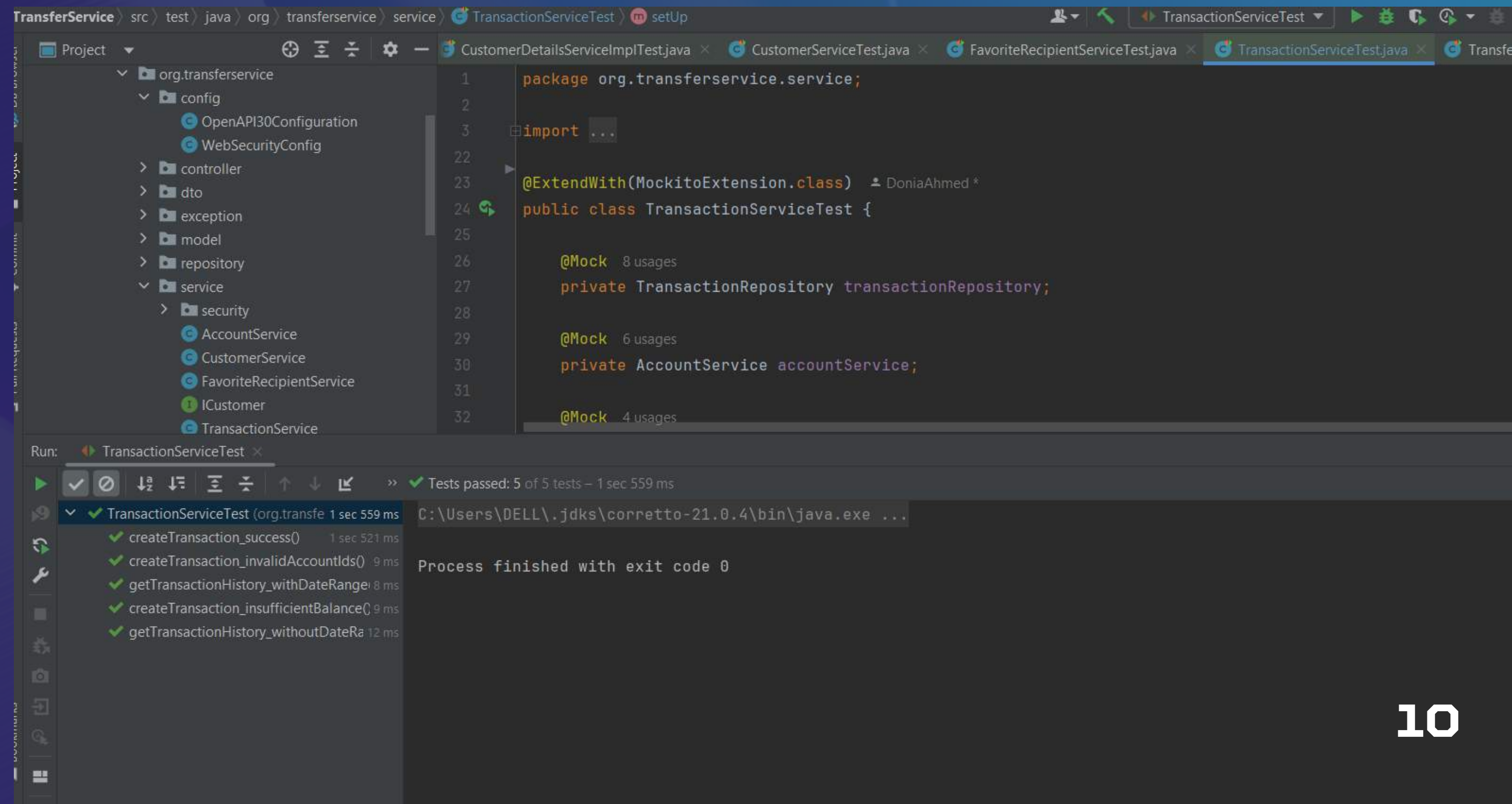
Process finished with exit code 0

- The test cases of the CustomerDetailsService has passed successfully here

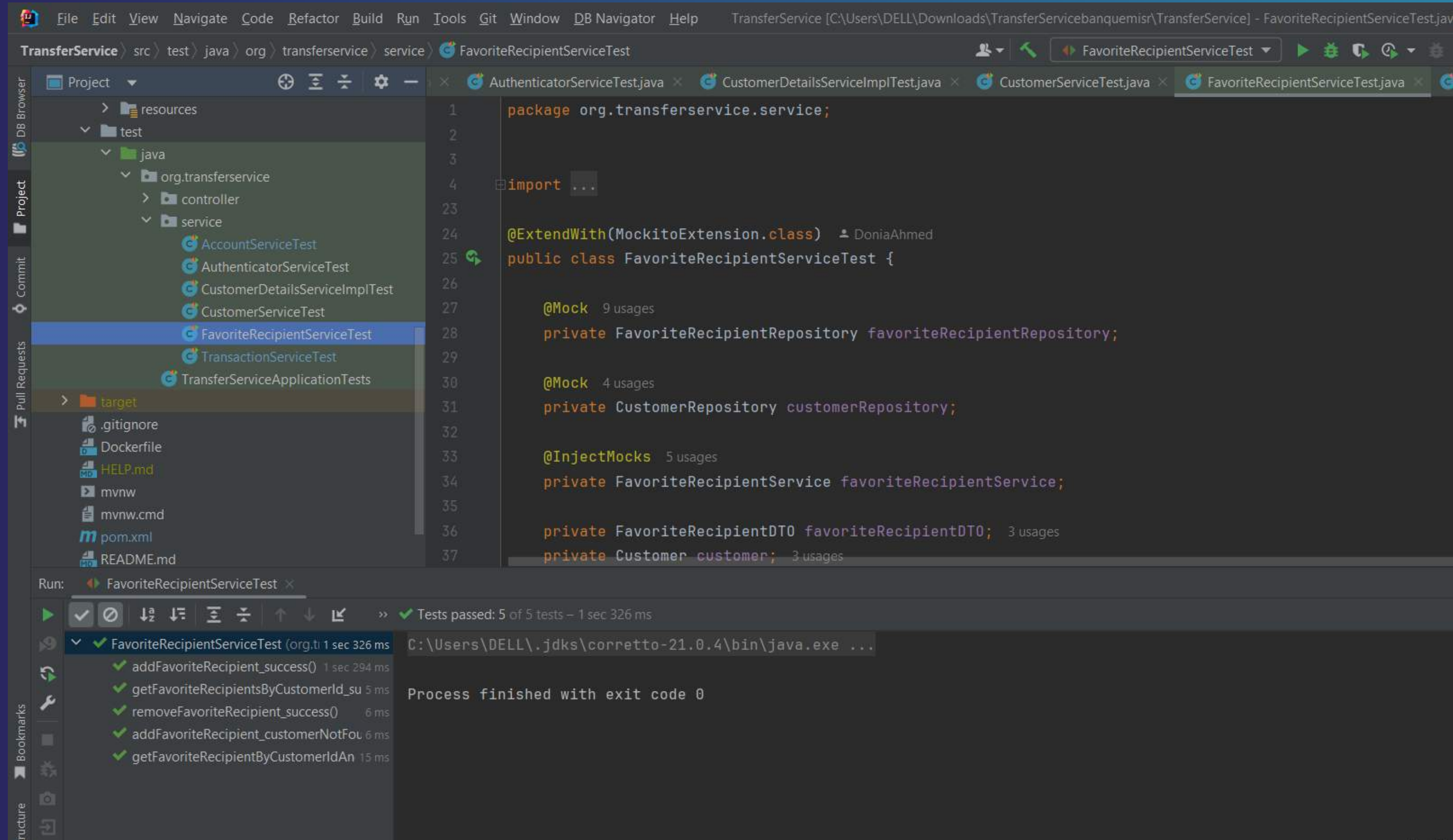
The screenshot displays an IDE with the following components:

- Project Explorer:** Shows the project structure with folders for `resources`, `test`, and `java`. The `java` folder contains `org.transferservice`, which has subfolders for `controller` and `service`. The `service` folder contains several test classes, with `CustomerServiceTest` selected.
- Code Editor:** Displays the `CustomerServiceTest` class. The code includes package declarations, imports, and annotations like `@ExtendWith(MockitoExtension.class)`, `@Mock`, `@InjectMocks`, and `@BeforeEach`. It defines private fields for `customerRepository`, `customerService`, `customer`, and `updateCustomerDTO`, and a `setUp()` method.
- Run Console:** Shows the execution results for `CustomerServiceTest`. It indicates that 6 out of 6 tests passed in 1 second and 823 milliseconds. The tests listed are `getCustomerById_success()`, `deleteCustomer_customerNotFound()`, `updateCustomer_customerNotFound()`, `getCustomerById_customerNotFound()`, `deleteCustomer_success()`, and `updateCustomer_success()`. The console also shows the command used to run the tests and the exit code.

- The test cases of the TransactionServiceTest has passed successfully as we see

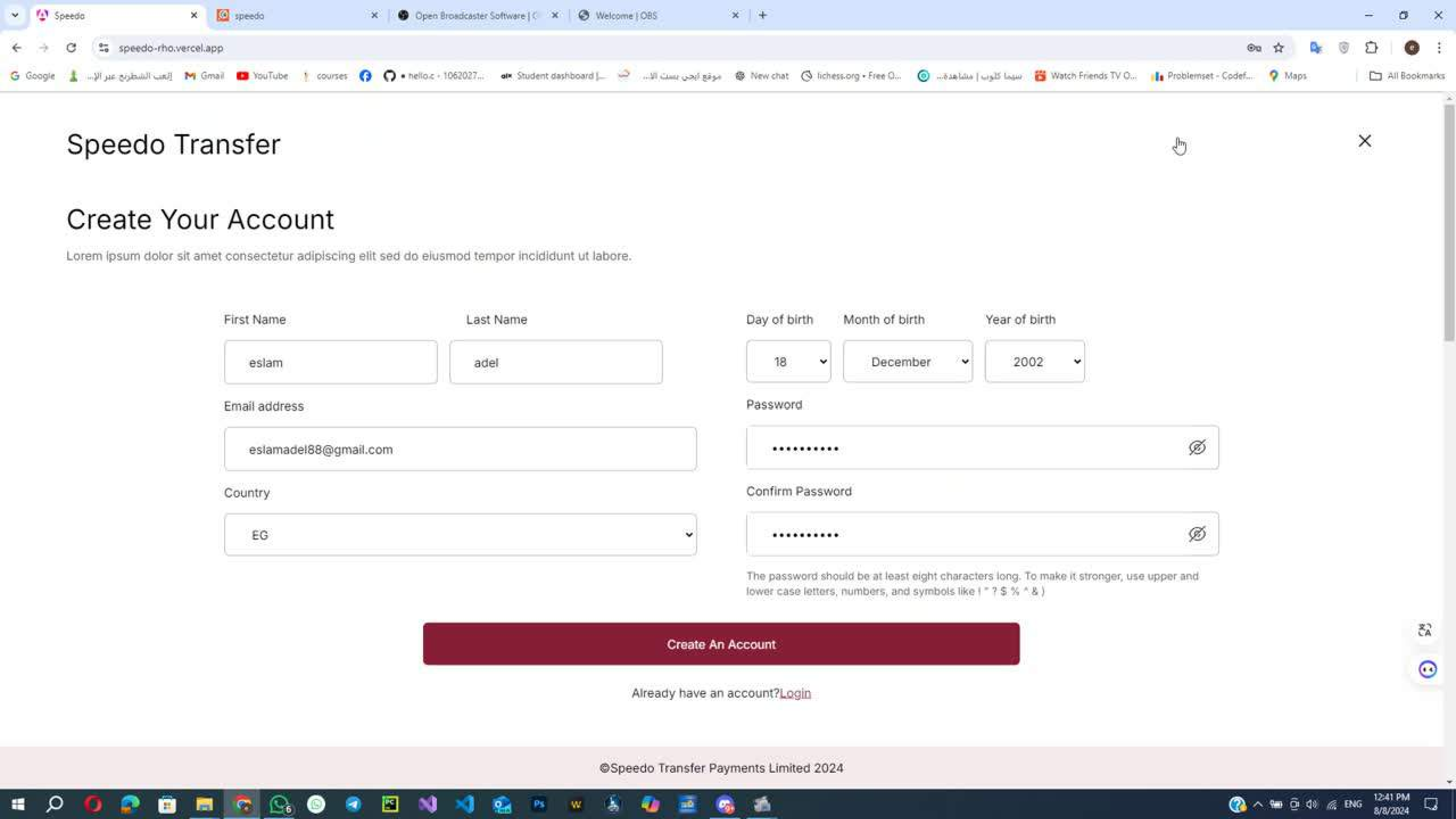


- The test cases of the FavoriteRecipientService has passed successfully as well



Front-End

- using last of version angular(standalone)
- Json sever(testing apis)
- services
- routes
- guards
- vaildations
- responsive(tailwind css)



Speedo Transfer

Create Your Account

Lorem ipsum dolor sit amet consectetur adipiscing elit sed do eiusmod tempor incididunt ut labore.

First Name

Last Name

Day of birth

Month of birth

Year of birth

Email address

Password

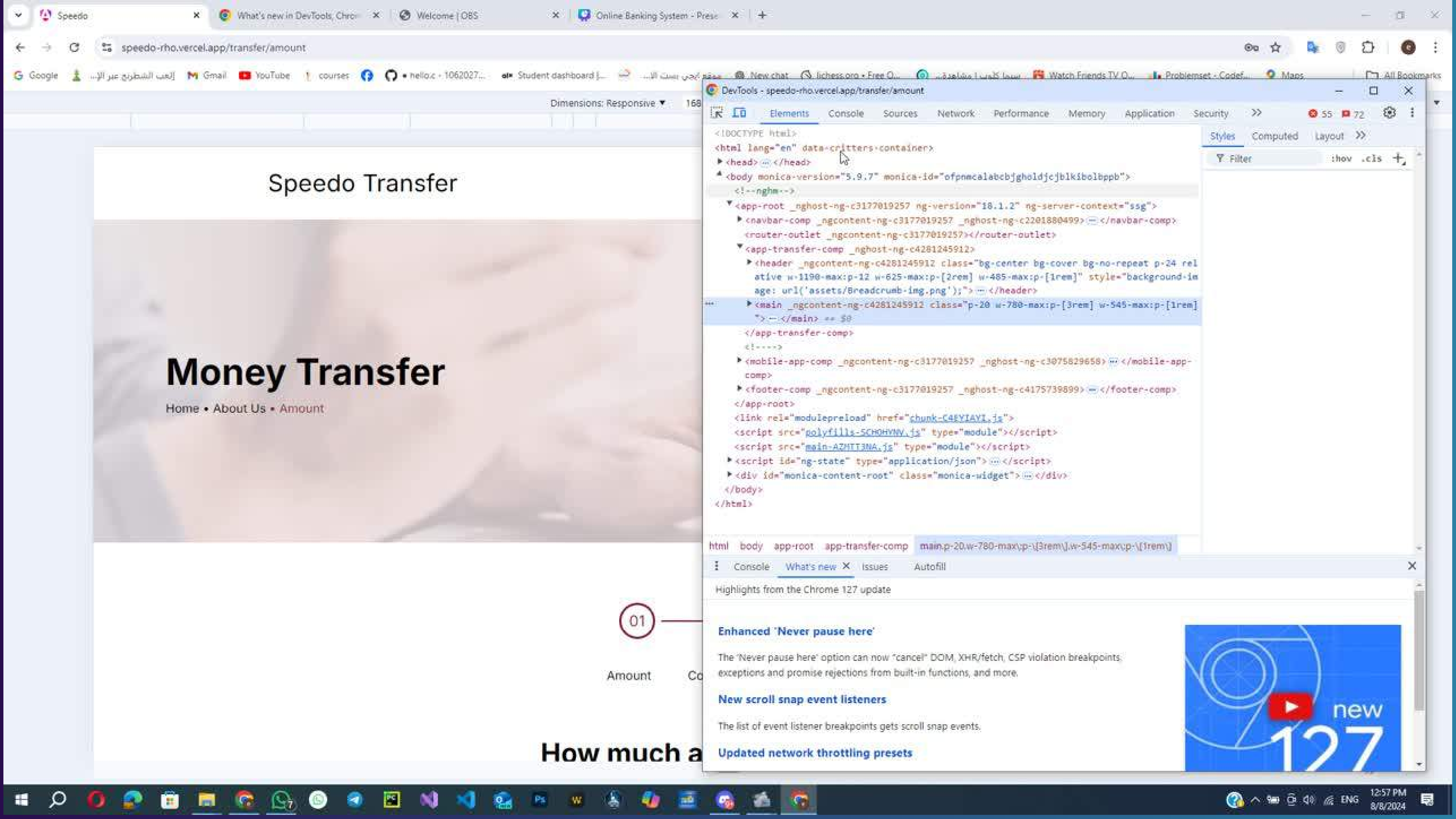
Confirm Password

Country

The password should be at least eight characters long. To make it stronger, use upper and lower case letters, numbers, and symbols like ! " ? \$ % ^ &)

Create An Account

Already have an account? [Login](#)



Speedo Transfer

Money Transfer

Home • About Us • Amount

DevTools - speedo-rho.vercel.app/transfer/amount

Elements Console Sources Network Performance Memory Application Security

```
<!DOCTYPE html>
<html lang="en" data-cr1tters="container">
  <head>...</head>
  <body monica-version="5.9.7" monica-id="ofpnmcalabcbjgholdjcjbikibolbpb">
    <!--ngm-->
    <app-root _ngghost-ng-c3177019257 ng-version="18.1.2" ng-server-context="ssg">
      <navbar-comp _ngcontent-ng-c3177019257 _ngghost-ng-c2201880499>...</navbar-comp>
      <router-outlet _ngcontent-ng-c3177019257></router-outlet>
      <app-transfer-comp _ngghost-ng-c4281245912>
        <header _ngcontent-ng-c4281245912 class="bg-center bg-cover bg-no-repeat p-24 rel
        ative w-1190-max:p-12 w-625-max:p-[2rem] w-485-max:p-[1rem]" style="background-im
        age: url('assets/Breadcrumb-1ng.png');">...</header>
        <main _ngcontent-ng-c4281245912 class="p-20 w-780-max:p-[3rem] w-545-max:p-[1rem]
        ">...</main> == $0
      </app-transfer-comp>
    <!--...-->
    <mobile-app-comp _ngcontent-ng-c3177019257 _ngghost-ng-c3075829650>...</mobile-app-
    comp>
    <footer-comp _ngcontent-ng-c3177019257 _ngghost-ng-c4175739899>...</footer-comp>
  </app-root>
  <link rel="modulepreload" href="chunk-C48YIAYI.js">
  <script src="polyfills-5CHOHYMW.js" type="module"></script>
  <script src="main-AZHTI3MA.js" type="module"></script>
  <script id="ng-state" type="application/json">...</script>
  <div id="monica-content-root" class="monica-widget">...</div>
</body>
</html>
```

html body app-root app-transfer-comp main.p-20.w-780-max;p-[3rem].w-545-max;p-[1rem]

Console What's new Issues Autofill

Highlights from the Chrome 127 update

Enhanced 'Never pause here'

The 'Never pause here' option can now "cancel" DOM, XHR/fetch, CSP violation breakpoints, exceptions and promise rejections from built-in functions, and more.

New scroll snap event listeners

The list of event listener breakpoints gets scroll snap events.

Updated network throttling presets



01
Amount Co

How much a

IOS Track

Used technology

- Alamofire
- FittedSheets
- IQ KeyBoardManager
- MVVM Architecture
- Networking Layer
- Routing Layer
- XIB

iPhone 15 Pro
00:1/1

1:25

Sign Up

Speedo Transfer

Full Name

John Doe

Email

john.doe@example.com

Password

SecurePassword123

Sign Up

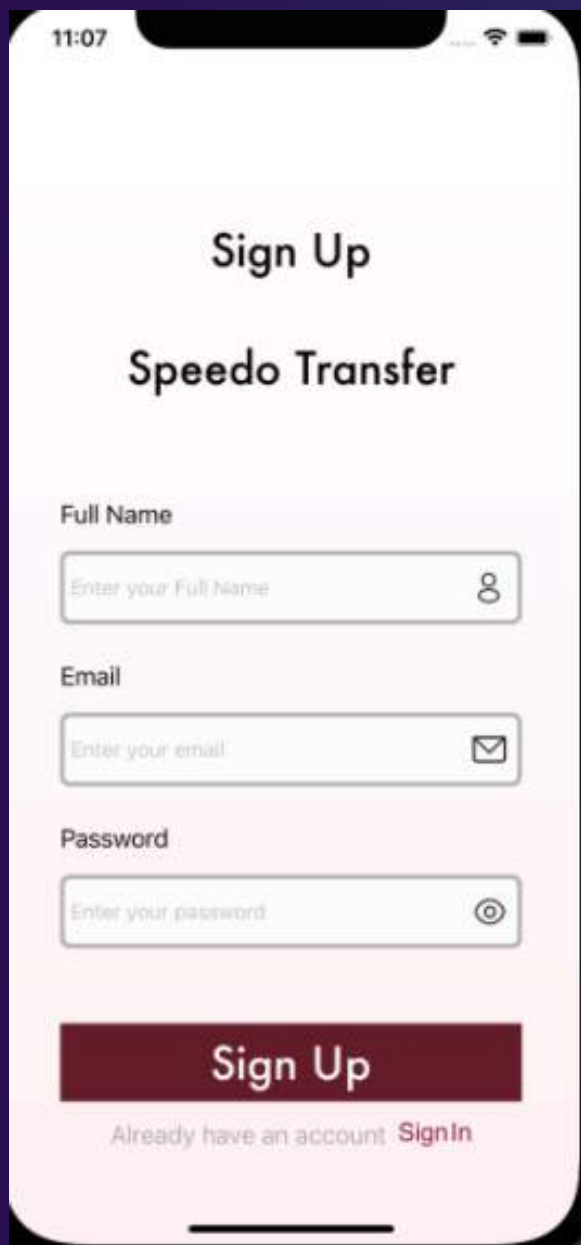
Already have an account?

Sign In

Enter your password

Done

Registration and login



11:07

Sign Up

Speedo Transfer

Full Name

Enter your Full Name

Email

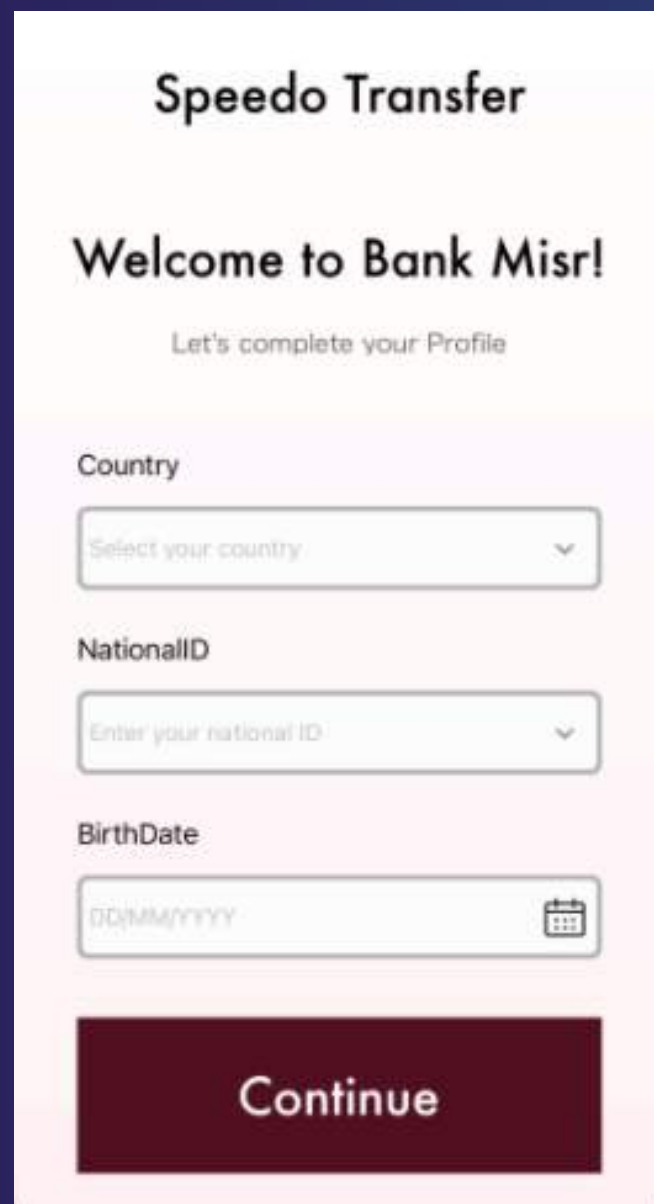
Enter your email

Password

Enter your password

Sign Up

Already have an account. [Sign In](#)



Speedo Transfer

Welcome to Bank Misr!

Let's complete your Profile

Country

Select your country

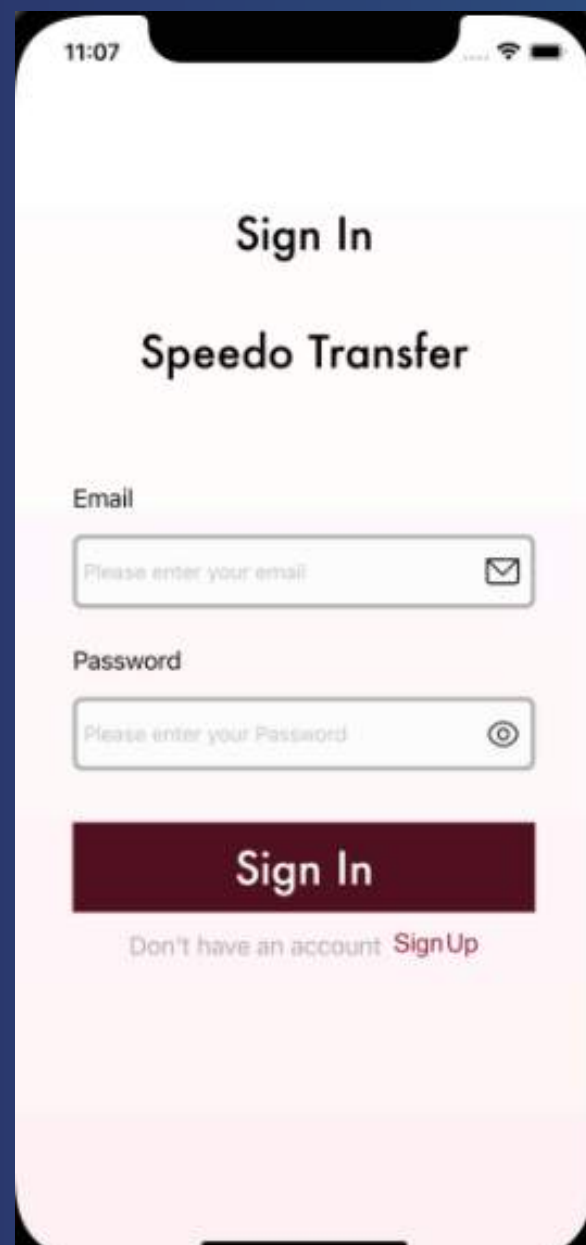
NationalID

Enter your national ID

BirthDate

DD/MM/YYYY

Continue



11:07

Sign In

Speedo Transfer

Email

Please enter your email

Password

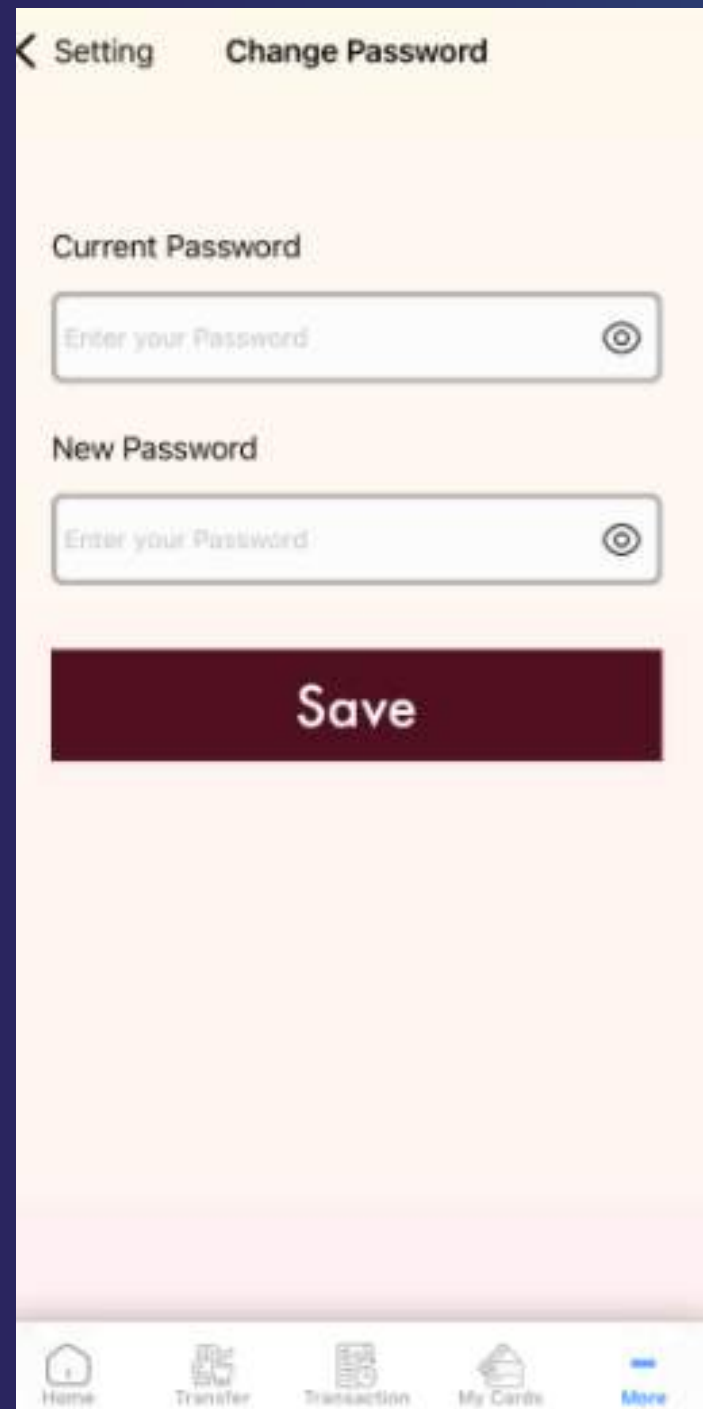
Please enter your Password

Sign In

Don't have an account. [Sign Up](#)

```
class SignUpAPIManager {
    static func signUp( user: UserModel, completion: @escaping(_ error: Error?, _
        status: Bool, _ message: String?) -> Void ) {
        do {
            let jsonData = try JSONEncoder().encode(user)
            var request = URLRequest(url: URL(string:
                "https://e8c8-156-223-19-253.ngrok-free.app/api/register")!)
            request.httpMethod = HTTPMethod.post.rawValue
            request.setValue("application/json", forHTTPHeaderField: "Content-Type")
            request.httpBody = jsonData
            AF.request(request).response {
                response in
                switch response.result {
                    case .success(let value):
                        if let code = response.response?.statusCode, code == 200 {
                            completion(nil, true, nil)
                            print(value ?? "cant be printed") // to get token
                            return
                        }
                        completion(nil, false, "Error in data decoding")
                    case .failure(let error):
                        completion(error, false, nil)
                }
            }
        } catch {
            completion(nil, false, "unknown error")
        }
    }
}
```

Change Password API



The image shows a mobile application interface for changing a password. At the top, there is a navigation bar with a back arrow and the text 'Setting' and 'Change Password'. Below this, there are two input fields: 'Current Password' and 'New Password'. Each field has a placeholder text 'Enter your Password' and a toggle icon on the right. At the bottom of the form is a large red button labeled 'Save'. The bottom of the screen features a navigation bar with icons for 'Home', 'Transfer', 'Transaction', 'My Cards', and 'More'.

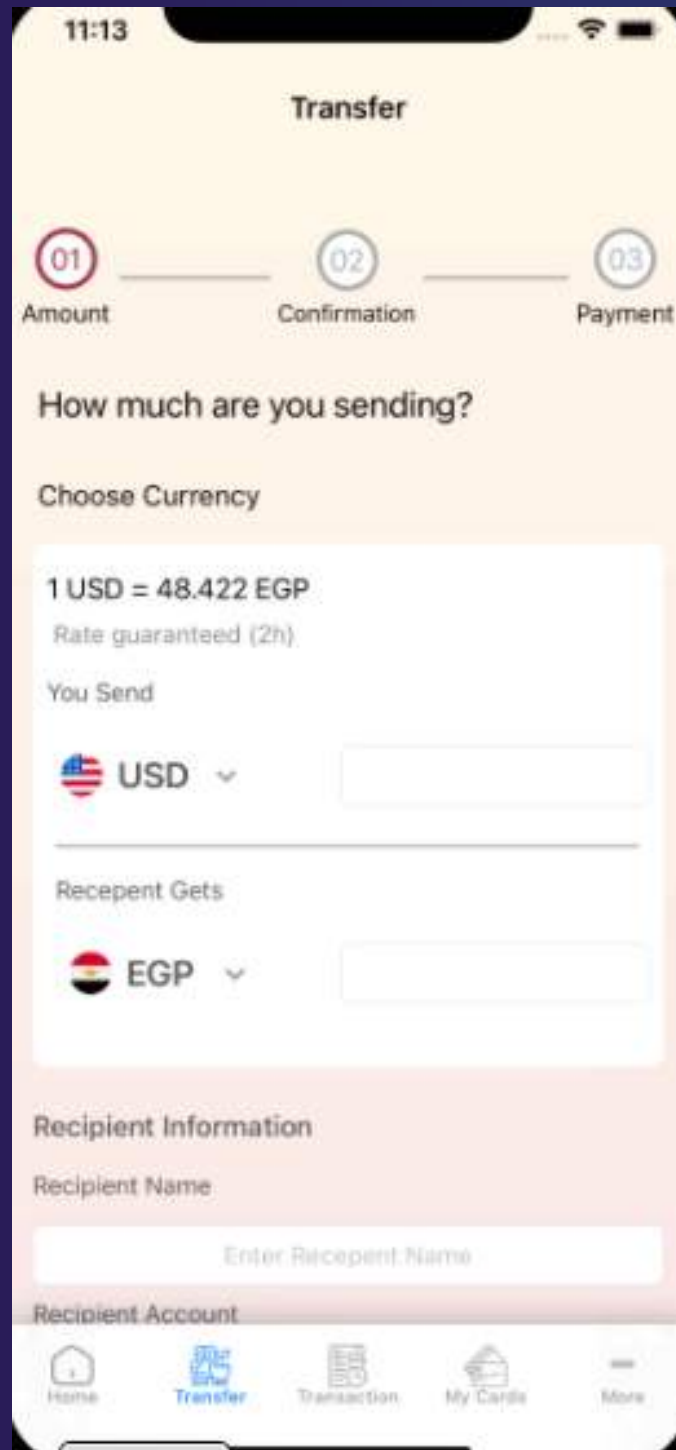
```
class ChangePasswordAPIManager {
    static func changePassword(oldPassword: String, newPassword: String, completion:
        @escaping (_ error: Error?, _ message: String?) -> Void) {
        guard let token = UserDefaultsManager.share().token else {
            completion(nil, nil)
            return
        }
        let headers: HTTPHeaders = [
            "Authorization": "Bearer \(token)",
            "Content-Type": "application/json"
        ]
        let body = [
            "oldPassword": oldPassword,
            "newPassword": newPassword
        ]
        AF.request("https://e8c8-156-223-19-253.ngrok-free.app/api/updatePassword",
            method: .put, parameters: body, encoder: JSONParameterEncoder.default,
            headers: headers).responseString { response in
                debugPrint(response)
                guard response.error == nil else {
                    debugPrint(response.error ?? "error response")
                    completion(response.error, nil)
                    return
                }
                switch response.result {
                case .success(let data):
                    print(data)
                    completion(nil, data)
                case .failure(let error):
                    completion(error, nil)
                }
            }
        }
    }
}
```

Select Currency API



```
8 import Foundation
9 import Alamofire
10 class GetCurrenciesAPIManager{
11
12     static func getCurrencies(completion: @escaping (_ error: Error?, _
13         countries: [Currency]?)->Void){
14         AF.request("https://restcountries.com/v3.1/all", method:
15             .get).response{ response in
16                 debugPrint(response)
17                 guard response.error == nil else {
18                     debugPrint(response.error ?? "error response")
19                     completion(response.error, nil)
20                     return
21                 }
22                 guard let data = response.data else {
23                     debugPrint("cant get data")
24                     completion(nil, nil)
25                     return
26                 }
27                 do {
28                     let currencies = try JSONDecoder().decode([Currency].self,
29                         from: data)
30                     completion(nil, currencies)
31                 } catch let error {
32                     print(error.localizedDescription)
33                     completion(error, nil)
34                 }
35             }
36     }
37 }
```

Currency exchange API



```
class GetChangeRateAPIManager {
    static func getExchangeRate(from baseCurrency: String, to targetCurrency:
    String, completion: @escaping (_ error: Error?, _ exchangeRate:
    Double?)->Void) {
        let urlString =
            "https://v6.exchangerate-api
            .com/v6/eaed7189a1c1e65d6113d216/latest/\(baseCurrency)"
        AF.request(urlString, method: .get).response { response in
            debugPrint(response)
            guard response.error == nil else {
                debugPrint(response.error ?? "error response")
                completion(response.error, nil)
                return
            }
            guard let data = response.data else {
                debugPrint("Can't get data")
                completion(nil, nil)
                return
            }
            do {
                let changeRate = try JSONDecoder().decode(ChangeRate.self, from:
                data)
                let exchangeRate = changeRate.conversionRates[targetCurrency]
                completion(nil, exchangeRate)
            } catch let error {
                print(error.localizedDescription)
                completion(error, nil)
            }
        }
    }
}
```

Android Team

As a user, I want to be able to create a new bank account with name, email, and a password. The password should be 6 characters at least with at least one capital letter, one small letter, and a special characters (e.g. \$%^)

Sing Up

Speedo Transfer

Full Name
Amr Anwar

Email
amr123@gmail.com

Password

Sign up

Already have an account? [Sing In](#)

Sing Up

Speedo Transfer

Full Name
Amr Anwar

Email
amr123@gmail.com

Password

Sign up

q w e r t y u i o p
a s d f g h j k l
z x c v b n m
?123 @ ☺ . ✓

Sing Up

Speedo Transfer

Full Name
Amr Anwar

Email
a
Wrong email format

Password
.
Password must be at least 6 characters long, contain at least one uppercase letter, one lowercase letter, and one special character.

Sign up

Already have an account? [Sing In](#)

Sing Up

Speedo Transfer

Full Name
Enter your Full Name

Email
Enter your email address

Password
Enter your password

Sign up

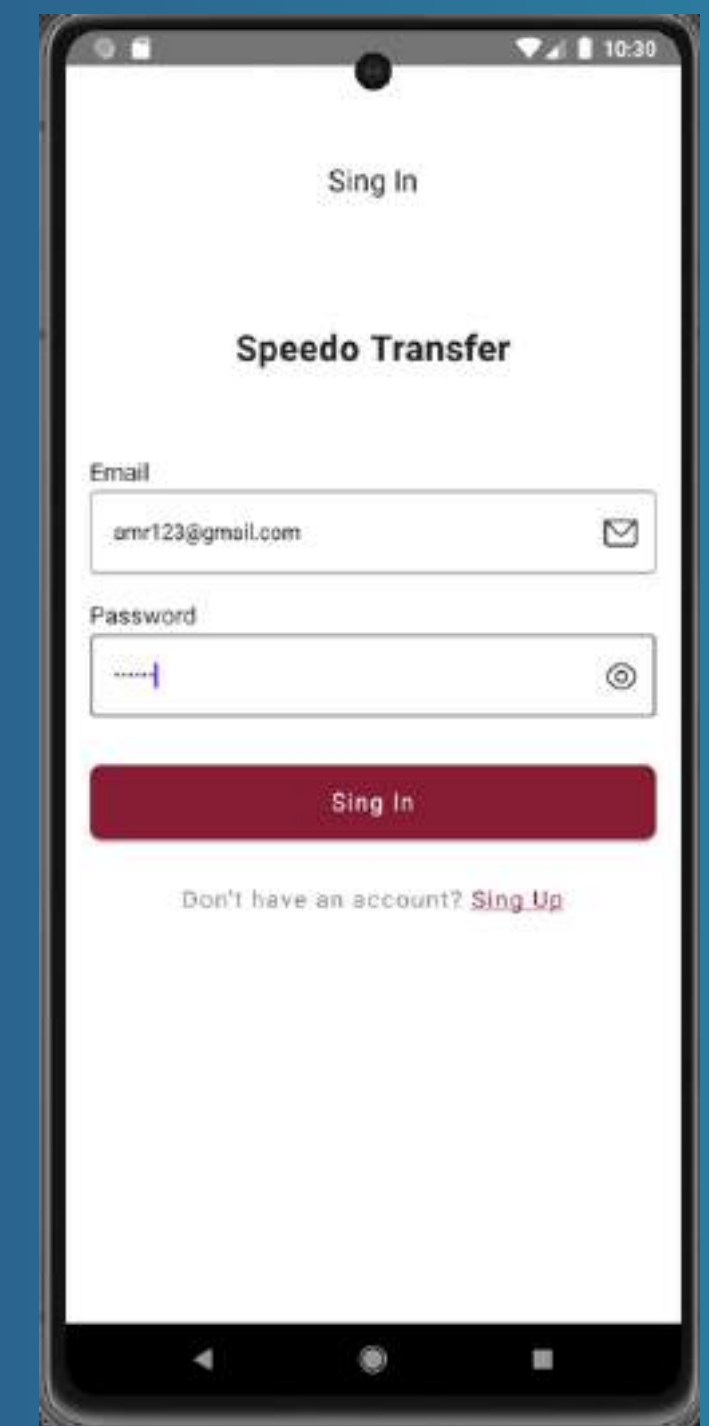
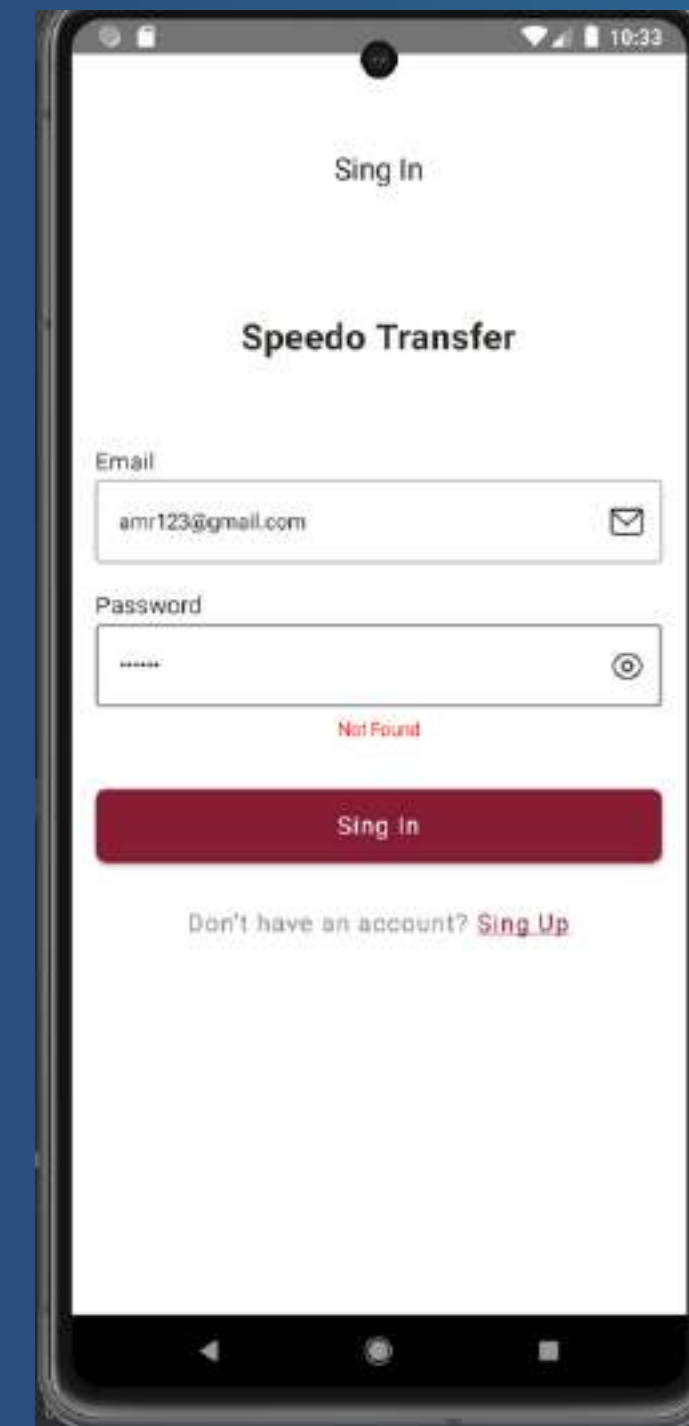
Already have an account? [Sing In](#)

As a user, I want to be able to login using my previously registered email & password to be able to use the full features of the application and when opening the application again, I want to find my previously written email and password shown in the text fields without the need to write them again

```
class SignIn : ComponentActivity() { @ Amr4x4 *
    override fun onCreate(savedInstanceState: Bundle?) { @ Amr4x4 *
        super.onCreate(savedInstanceState)
        val repository = UserRepository(RetrofitClient.apiService)
        val viewModel = UserViewModel(repository)

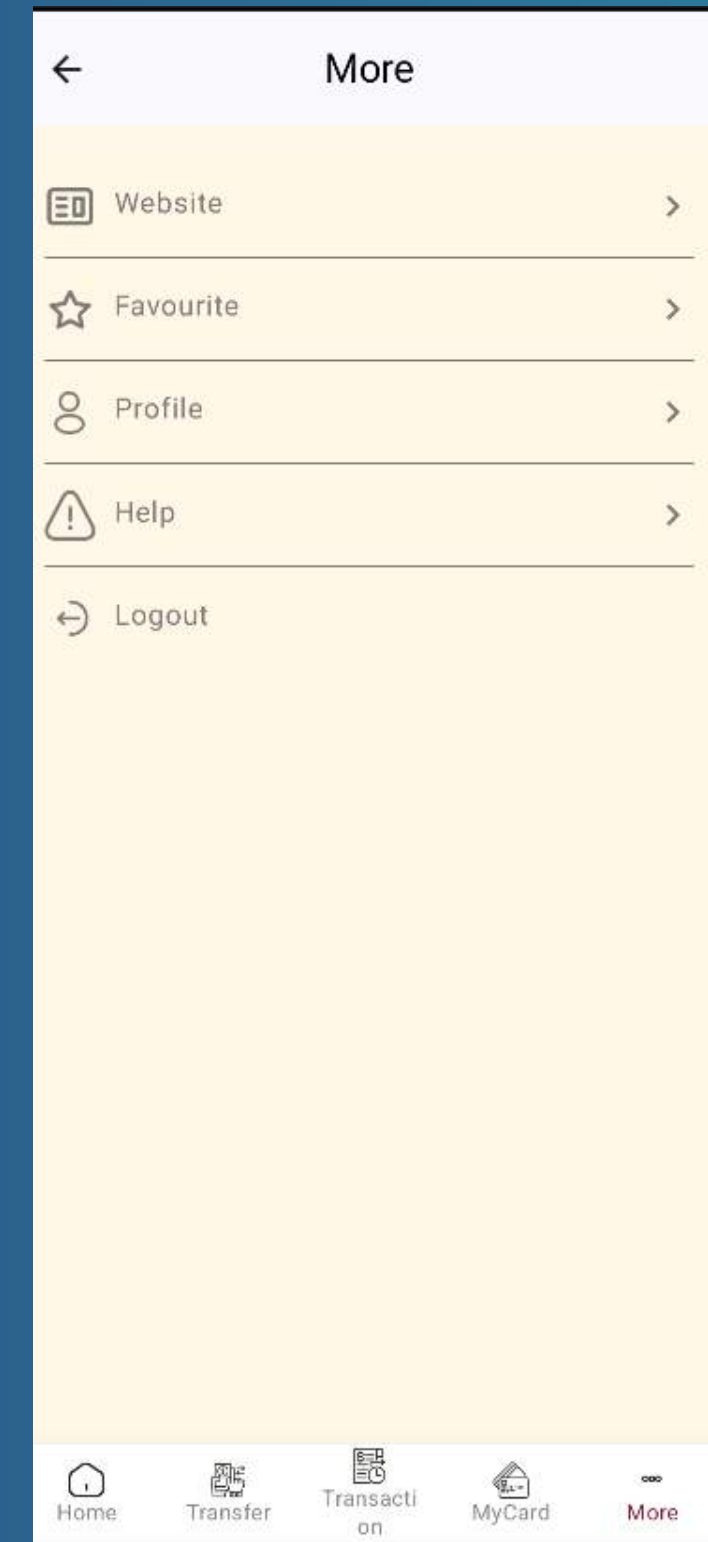
        val sharedPreferences = getSharedPreferences(name: "SpeedoTransferPrefs", Context.MODE_PRIVATE)

        setContent {
            SpeedoTransferTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colors.background
                ) {
                    SignInScreen(viewModel = viewModel, sharedPreferences = sharedPreferences)
                }
            }
        }
    }
}
```



As a user, I want to logout to be able to login using another account or to register a new one.

```
{  
    val intent = Intent(context, SignUpFirstScreen::class.java).apply {  
        // Clear the back stack and start new task  
        flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK  
    }  
    context.startActivity(intent)  
}
```



As a user, I should receive an alert dialog after 30 minutes of inactivity (e.g. didn't request or send anything from/to the server) that takes me to login again

```
var showDialog by remember { mutableStateOf( value: false) }

val resetTimer = InactivityTimer(ontimeout = {
    showDialog = true
})

CompositionLocalProvider( value: LocalInactivityTimer provides resetTimer) {
    InactivityAlertDialog(
        showDialog = showDialog,
        onDismiss = { showDialog = false },
        onConfirm = {
            showDialog = false
            // Navigate to login screen
            navController.navigate( route: "login_screen") {
                popUpTo( route: "main_screen") { inclusive = true }
            }
        }
    )
}
```

```
val localInactivityTimer = compositionLocalOf<() -> Unit> { error("No timer provided") }

@Composable new*
fun InactivityTimer(ontimeout: () -> Unit): () -> Unit {
    var lastActivityTime by remember { mutableStateOf(System.currentTimeMillis()) }

    LaunchedEffect(Unit) {
        while (true) {
            delay( timeMillis: 1000) // Check every second
            val currentTime = System.currentTimeMillis()
            if (currentTime - lastActivityTime >= 30 * 60 * 1000) { // 30 minutes
                ontimeout()
                break
            }
        }
    }

    // Function to reset the timer
    fun resetTimer() {
        lastActivityTime = System.currentTimeMillis()
    }

    return ::resetTimer
}
```

As a user, I want to transfer money to other accounts with a limit of 5000 L.E. per transaction. And before the transaction is made, I want to make sure that the other account details are valid to avoid making invalid transactions and that the transaction doesn't exceed my current balance along with a a notification after a successful transaction.

```
if (sendAmount.text.toDoubleOrNull() ?: 0.0 > 5000 && isUsdToEgp) {  
    Text(  
        text = "You can transfer money to other accounts with a limit of 5000 L.E. per transaction.",  
        color = Color.Red,  
        fontSize = 12.sp,  
        modifier = Modifier.padding(top = 8.dp)  
    )  
}  
}
```

< Transfer

01 02 03
Amount Confirmation Payment

How much are you sending?

Choose Currency

1 USD = 48.4220 EGP
Rate guaranteed (2h)

You Send

USD 4000

Recipient Gets

EGP 193688.0

Recipient Information ☆ Favorite >

Recipient Name

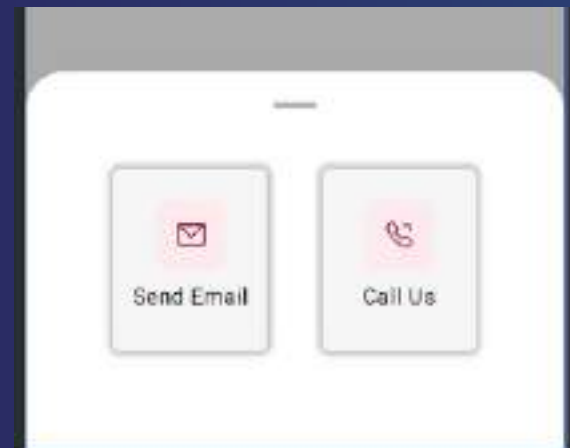
Enter Recipient Name

Recipient Account

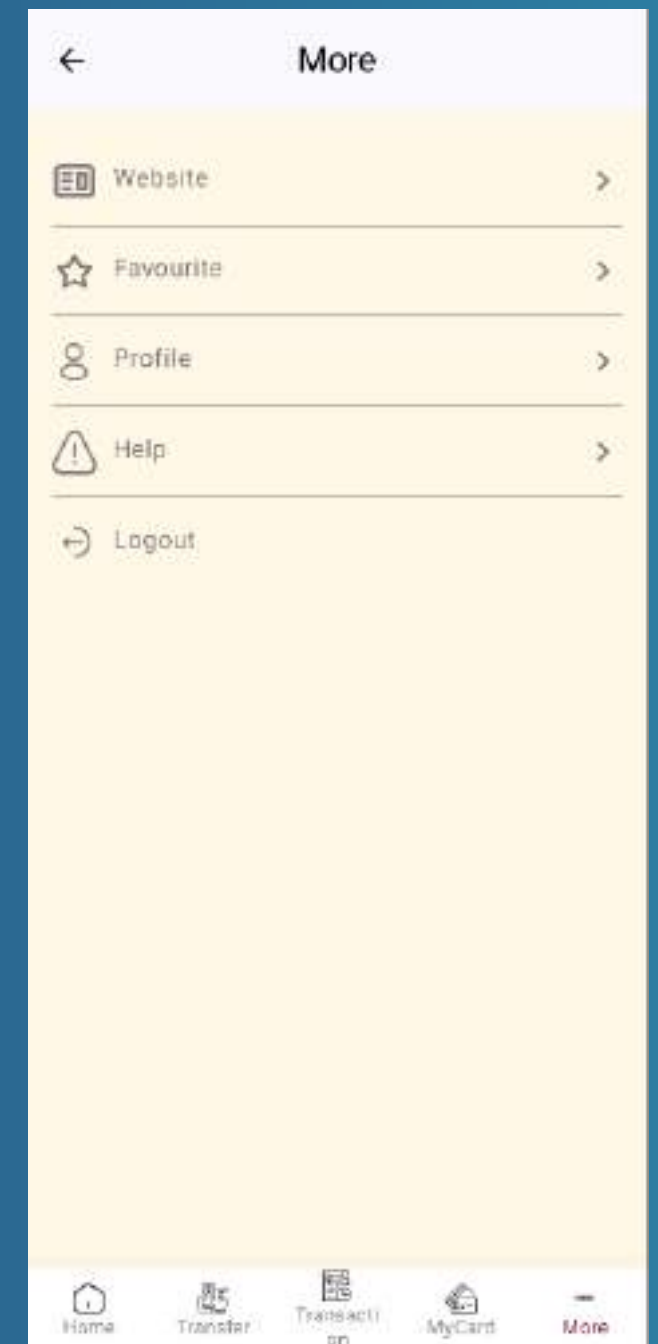
Enter Percipient Account Number

Home Transfer Transact MyCard More

As a user, if there is an issue in the app, I would like to have two options: a. Call the support b. Send an email to the support

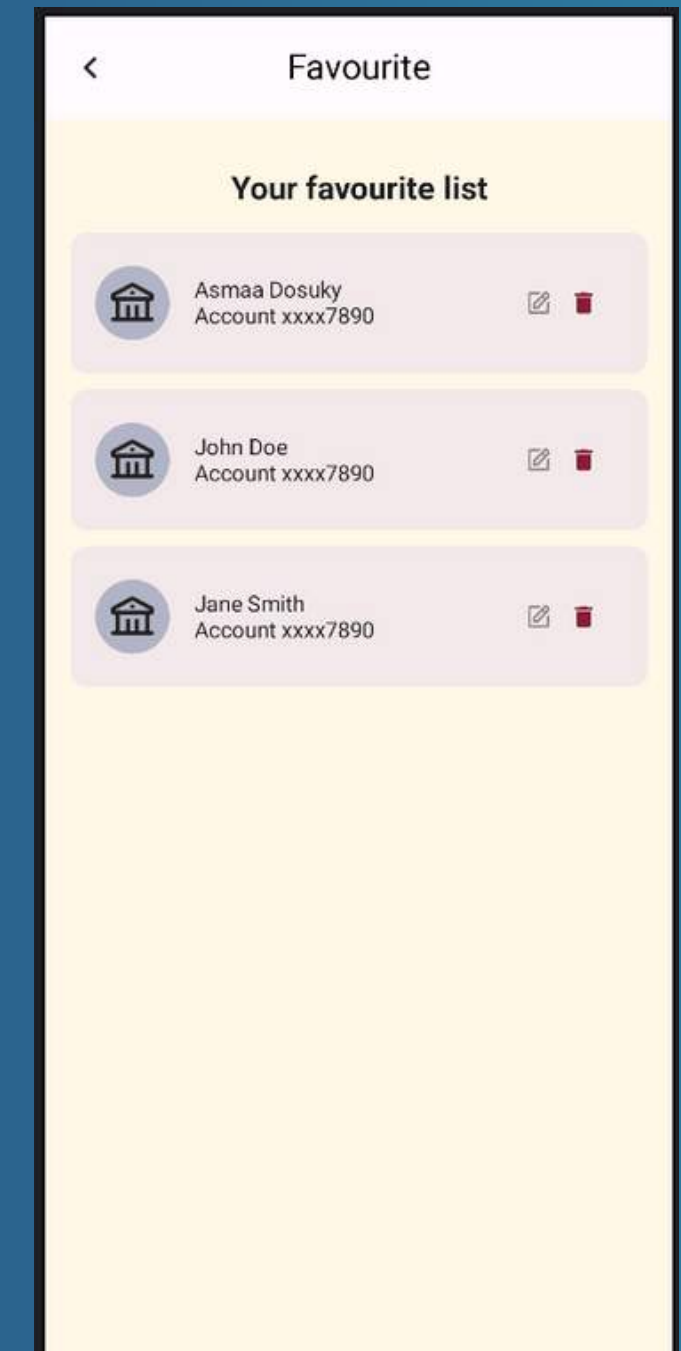


```
HelpItem(  
    icon = R.drawable.email,  
    text = "Send Email",  
    onClick = {  
        val intent = Intent(Intent.ACTION_SENDTO).apply {  
            data = Uri.parse(uriString: "mailto:help@speedo.com")  
        }  
        context.startActivity(intent)  
    },  
)  
Spacer(modifier = modifier.width(32.dp))  
HelpItem(  
    icon = R.drawable.call,  
    text = "Call Us",  
    onClick = {  
        val intent = Intent(Intent.ACTION_DIAL).apply {  
            data = Uri.parse(uriString: "tel:000000")  
        }  
        context.startActivity(intent)  
    },  
)
```



As a user, I want to save one or more recipients in my favorites, to be able to retrieve them anytime live without the need to refresh the list for quick money transfers and to delete a specific recipient from the favorites if want to

```
favouriteItems.forEach { item ->
    FavouriteItem(
        name = item.name,
        accountNumber = item.accountNumber,
        onEditClick = {
            selectedItem = item
            recipientName = item.name
            recipientAccount = item.accountNumber
            isEditSheetOpen = true
            scope.launch { sheetState.show() }
        },
        onDeleteClick = {
            favouriteItems = favouriteItems.filter { it != item }
        }
    )
}
```



Thanks alot for Your time :)